

LAAKA: Lightweight Anonymous Authentication and Key Agreement Scheme for Secure Fog-Driven IoT Systems

Hala Ali *, Irfan Ahmed

Department of Computer Science, Virginia Commonwealth University, Richmond, 23284, VA, United States

ARTICLE INFO

Keywords:

Fog Computing
IoT
Lightweight Authentication
Key Agreement
BAN Logic
Random Oracle Model
Scyther

ABSTRACT

Fog computing is a new computing model with limited resources located near the Internet of Things (IoT) devices to provide low transmission delay, mobility, and location awareness services. However, fog technology not only extends the cloud resources but also inherits its properties and security problems, such as device capture, man-in-the-middle, secret key compromise, identity impersonation, and message replay attacks. These vulnerabilities pose significant risks to the integrity of IoT-fog communications. Nevertheless, the existing mutual authentication approaches either suffer from high computational overheads or fail to provide secure mutual authentication. To address this gap, this paper introduces LAAKA, a Lightweight Anonymous Authentication and Key Agreement scheme. Considering the constrained resources of IoT and fog devices, LAAKA utilizes lightweight operations such as hash function and bitwise XOR. Its main objective is to facilitate mutual authentication and establish secure session keys between IoT devices and fog servers, making it useful for various IoT applications. We validate the robustness of LAAKA against various security threats by conducting comprehensive formal security analysis, including Burrows-Abadi-Needham (BAN) logic and Random Oracle Model (ROM), as well as informal analysis. The efficacy of our scheme is further demonstrated through evaluation with the Scyther tool. Compared to other proposed authentication approaches, the results illustrate the superior performance and efficiency of our approach in enhancing the security features, minimizing the computational cost, and optimizing storage utilization.

1. Introduction

The rapidly increasing number of Internet of Things (IoT) devices results in generating massive amounts of data (Guo et al. (2020)). Due to the limited processing capacity of IoT devices, this generated data is usually submitted to the cloud servers for processing. Cloud computing has high processing capacity and unbounded storage resources to execute the assigned tasks and store the received data efficiently (Almurisi and Tadisetty (2022); Ali and Sridevi (2023)). However, the cloud nodes are located physically far from the IoT data sources, leading to high network latency and thus affecting the response time of the IoT application (Ali et al. (2021a)). Moreover, the centralized processing of tasks by the cloud makes it an unsuitable and inefficient computing paradigm for processing location-aware and real-time applications. Thus, a new computing technology was introduced by Cisco in 2012 (Jiang et al. (2020)) called fog computing to overcome the limitations of the cloud and shift its resources to the network edge near the IoT data sources. The features of fog computing help in reducing the com-

munication delay and supporting mobility along with the geographical distribution of the IoT devices while serving them (Rana et al. (2021b)).

Despite fog computing being more secure than the cloud and providing a wide range of services efficiently, this computing paradigm still faces many security issues that should be mitigated (Guo et al. (2020)). Authentication of IoT and fog devices is considered one of the most critical security issues in fog architecture that should be considered. However, authentication of devices faces many challenges, such as location tracking (Nandy et al. (2019)), impersonating the device identity (Tu et al. (2018)), and capturing the device physically (Ometov et al. (2022)). These challenges result from insufficient protection for the distributed IoT and fog devices and their constrained resources, which leads to penetration, destruction, and stealing of these devices easily (Roman et al. (2018)). Therefore, the following security requirements should be addressed in fog computing:

- Authentication: This essential security service prevents the attacker from pretending to be a legitimate IoT or fog device. However, the

* Corresponding author.

E-mail addresses: alih16@vcu.edu (H. Ali), iahmed3@vcu.edu (I. Ahmed).

limited computational capacity of the IoT and fog devices is a barrier to implementing authentication in the IoT-fog environment. So, traditional methods such as public key cryptography-based methods are not preferred to be used (Mukherjee et al. (2017)).

- Anonymity: This security service is necessary to hide the identity of the devices during the authentication phase to prevent the device tractability by the attacker (Kaur et al. (2020)).
- Forward and backward secrecy: This service ensures that if an attacker somehow obtains the current session key, it cannot be used to decrypt previous messages or future communications. (Weng et al. (2021); Kunal et al. (2019)).
- Secure communication: The transmitted sensitive IoT data must be confidential and accessible by authorized devices only. This service is achieved by encrypting the exchanged data over the wireless communications between the IoT devices and fog nodes, which can be eavesdropped easily (AbuTaha et al. (2011)).
- Independency of session keys: This service ensures that compromising a session key does not compromise other established session keys, as it generates distinct and random session keys for each device (Ibrahim (2016)).

As automation becomes increasingly integral across diverse domains, including smart industrial settings (Panda et al. (2022)), smart home (Jan et al. (2022)), and healthcare systems (Chen et al. (2022)), the capability of embedded systems to autonomously gather and transmit real-time data for intelligent decision-making has become crucial. This automation, which often involves direct communication between IoT devices and local fog or cloud servers, is carried out with minimal human interference. In this context, the communication link from IoT to server is critical, enabling direct machine-to-machine interactions and playing a main role in the automation environments. Ensuring the authenticity of communication partners is a key security concern in these IoT-fog communication frameworks, serving as a countermeasure against different security threats like replay, man-in-the-middle, impersonation, and denial-of-service (Wazid et al. (2020)).

The absence of robust authentication could allow malicious actors to disrupt the decision-making process, for instance, by feeding erroneous data to the controllers in an Industry 4.0 setting. However, the application of intensive cryptographic operations for authentication is impractical for IoT devices, which are constrained by their processing power, communication bandwidth, storage capacity, and energy resources (Jan et al. (2022)). This requires the adoption of a mutual authentication and key agreement protocol that is not only lightweight but also efficient, ensuring the preservation of security within IoT-fog communication networks.

Despite the numerous mutual authentication and key agreement protocols designed to secure IoT-fog communication, a majority of these protocols encounter several challenges. These include vulnerability to Ephemeral-Secret-Leakage (ESL) attack (Guo et al. (2020); Jia et al. (2019); Zhang et al. (2020)), Denial of Service (DoS) attack (Vinoth et al. (2020); Guo and Guo (2021); Wazid et al. (2019b)), susceptibility to device capture attack (Wang et al. (2020); Vinoth et al. (2020); Wazid et al. (2019b)), and a lack of user and device anonymity (Ibrahim (2016); Ma et al. (2019); Li et al. (2020); Das et al. (2019)). Further issues include the absence of Perfect Forward Secrecy (PFS) (Kalaria et al. (2021)), the risk of user and device traceability attack (Ibrahim (2016); Wazid et al. (2019b); Ma et al. (2019); Kalaria et al. (2021)), high communication latency (Hamada et al. (2022); Weng et al. (2021); Singh and Chaurasiya (2021)) and high computational cost (Wang et al. (2020); Amor et al. (2017); Wu et al. (2021b); Lansky et al. (2022)). Moreover, many of these protocols depend on the presence of a registration authority to establish and distribute the session keys between the IoT devices and fog servers (Rahman and Wen (2019); Weng et al. (2021); Wu et al. (2021a)), which can be a significant limitation.

In contrast, some authentication protocols are designed based on lightweight operations, such as hash function and bitwise XOR, con-

sidering the limited resources of IoT and fog servers. However, these approaches also come with their own limitations. For instance, the protocol proposed by Abdussami et al. (2022) requires a cloud server for mutual authentication between IoT devices and fog servers, while Masud et al. (2021)'s scheme is vulnerable to key impersonation, masquerade gateway, and privileged insider attacks. Similarly, Suganthi et al. (2020)'s approach is vulnerable to impersonation and privileged insider attacks, while Dhillon and Kalra (2017)'s approach is not resistant against user impersonation, and stolen mobile device attacks. It also does not provide user anonymity and session key agreement.

Although the literature proposes various authentication and key agreement protocols, a fully effective solution for these vulnerabilities is still absent. Moreover, given the constrained computational, storage, and power capabilities of IoT and fog devices, designing an authentication scheme that is both resource-efficient and secure remains a challenge. This gap in the field has motivated us to develop a robust, lightweight mutual authentication and key agreement protocol that overcomes these vulnerabilities and ensures secure communication between IoT devices and fog servers.

Therefore, in this paper, we aim to leverage the primary advantage of fog computing, which is minimizing the network delay for IoT data transmission. To this end, we strategically place the trusted party registration authority within the fog layer, close to the IoT devices and fog servers, rather than in the cloud. This setup leads us to introduce LAAKA, a Lightweight Anonymous Authentication and Key Agreement scheme. LAAKA is designed to address the vulnerabilities in existing authentication schemes by ensuring robust mutual authentication between IoT devices and fog servers. It facilitates creating secure session keys, generated through distinct random numbers, without the need to involve the registration authority. Furthermore, this scheme maintains low computational and storage requirements by employing only lightweight operations, such as hash functions and bitwise XOR.

We summarize the contributions of this work as follows:

- Propose an efficient lightweight authentication scheme between the IoT devices and fog servers, using hash function and bitwise XOR operations.
- Implement the authentication mutually between the IoT devices and fog servers without the help of the Registration Authority.
- Establish a distinct secret session key for each connection of the IoT device with the fog server to secure their transmissions.
- Analyze the scheme's performance using BAN logic, Random Oracle Model (ROM), and Scyther tool, then compare it to the performance of other existing authentication schemes in terms of security features, computational cost, and storage requirements.

The remainder of the paper is sketched as follows: Section 2 introduces some related works. Section 3 presents the system architecture and threat models. Section 4 discusses the proposed authentication and key agreement scheme (LAAKA), while its security analysis (i.e., formal and informal) is conducted in Section 5. Section 6 shows the performance analysis of our scheme and the results of comparison with other presented schemes. Finally, the work is concluded in Section 7 in addition to some suggestions for future directions.

2. Related work

This section shows some recently proposed mutual authentication and key agreement schemes in fog computing. Ali and Sridevi (2022) designed an authentication mechanism that enables the IoT device to mutually authenticate with the fog server, using hash function and asymmetric key cryptography. However, this mechanism causes a high computational cost to achieve anonymity and provide secure transmission between the IoT and fog devices. Ibrahim (2016) presented a secure authentication protocol to enable the user and fog server to authenticate each other with cooperation from the cloud as the registration

authority. This protocol also adopted hash function and asymmetric encryption. Although the proposed work reduces the computational and storage cost, it does not provide identity anonymity, exposing it to the man-in-the-middle attack. Furthermore, the secret keys are stored in clear form in the user device's memory and also in the fog server's memory, causing the device capture attack to access this confidential information. These limitations have been solved with the proposed authentication approach by Rahman and Wen (2019), using hash-based message authentication code along with symmetric key encryption. Despite this approach being protected from the man-in-the-middle attack and providing the user identity anonymity, there are no details about how the secret key was shared previously between the fog user and the registration authority. Moreover, establishing the session key requires the existence of the cloud as the registration authority, leading to more communication overhead and high network latency. One more disadvantage of this approach is that the long-term secret key is stored in clear form in the memory of the fog and user devices and used to generate the session key. In contrast, Hamada et al. (2022) enhanced this work by establishing the session key without resorting to the registration authority, using ECC operation and symmetric key encryption. However, this scheme is limited in the Canetti-Krawczyk (CK) threat model, where the secret keys can be extracted from the memory of the compromised or stolen user or fog device.

Wang et al. (2020) also used hash function and symmetric encryption in their proposed lightweight authentication scheme to generate dynamic session keys. However, many keys are used in this protocol while transmitting the authentication messages, causing high computational overhead and time consumption. Additionally, this protocol is not resistant to the device capture attack. The same cryptographic operations have been used in the scheme proposed by Vinoth et al. (2020) to implement multi-factor mutual authentication between the users and sensors in an industrial IoT environment. However, their scheme is insecure against device capture, denial of service, and replay attacks. Amor et al. (2017) designed a mutual authentication protocol in fog architecture, considering techniques such as using hash, ECC, and bilinear pairing. This protocol is immune to replay, man-in-the-middle, and eavesdropping attacks. In contrast, it causes a high computational cost and suffers from session key compromising and fog server impersonation attacks.

Lightweight authentication schemes often focus on the use of hash functions and bitwise XOR operations for enhanced efficiency. Weng et al. (2021) developed mutual authentication and key agreement between users and edge devices with fog servers, utilizing hash function and bitwise XOR operation to reduce computational costs and enhance security. However, this scheme involves the cloud registration authority in session key generation, leading to increased load on the cloud, potential exposure of confidential data, and added network delays. Additionally, the absence of timestamps in message exchanges makes it susceptible to replay attack. Similarly, Rana et al. (2021a) employed hash functions and bitwise XOR in developing a lightweight authentication protocol for IoT devices. As analyzed by Das et al. (2021), this protocol fails to efficiently protect the session key, does not offer user anonymity, presents a risk of user traceability, and is vulnerable to ESL attack. Guo and Guo (2021) also adopted hash functions and XOR operations in their protocol for anonymous authentication and key agreement between mobile IoT devices and fog nodes. Despite its benefits, this protocol faces several challenges, such as vulnerability to DoS attack, potential for user traceability, risk of offline password guessing, and insufficient security for the shared session key.

Elliptic Curve Cryptography (ECC) is widely used in fog computing authentication schemes along with hash functions and bitwise XOR operations due to its suitability for devices with limited resources. Wazid et al. (2019b) presented a user authentication scheme called SAKA-FC, using secure hash function, bitwise XOR, and ECC operation. Although the SAKA-FC scheme provides robust security functionalities and low computational cost, it is still insecure against clogging, user imperson-

ation, traceability, and device capture attacks. Ali et al. (2021b) presented an authentication approach for smart devices in fog computing to overcome the vulnerabilities of Wazid et al. (2019b)'s scheme. Although this approach is more secure and robust against many attacks, it causes more computational cost and high delay due to involving the cloud server in the proposed approach. Patonico et al. (2019) also proposed an ECC-based protocol in fog computing, focusing on mutual authentication and key agreement. This protocol ensures identity anonymity and secure session keys under the Canetti-Krawczyk (CK) threat model, but it doesn't protect against replay and device capture/stolen attacks. Singh and Chaurasiya (2021) presented another lightweight authentication scheme using hash function and ECC operation. Despite its low computational cost, this scheme is vulnerable to internal spoofing attacks in fog-cloud computing and requires the cloud as part of the authentication procedure, leading to a high load on the cloud and network latency.

Various studies have focused on developing mutual authentication schemes independent of third-party involvement. Guo et al. (2020) presented an efficient authentication scheme that excludes cloud servers as a third party. This scheme employs hash functions, symmetric polynomials, and bitwise XOR operations, yet it remains vulnerable to ESL attack as per the Canetti-Krawczyk adversary model. Similarly, Gope (2019) introduced three privacy-preserving lightweight authentication protocols in fog-cloud environment. These protocols are applicable in three distinct scenarios: authentication of end devices by a centralized server, authentication among authenticated end devices themselves, and authentication of end devices by network access devices or base stations. Utilizing lightweight cryptographic techniques, including hash functions, bitwise XOR, and symmetric encryption (AES), these schemes offer operational efficiency. However, they exhibit vulnerabilities to several security threats, such as capturing the end device or network access device, known session key, masquerading, and DoS attacks in addition to the absence of perfect forward secrecy. Kalaria et al. (2021) also proposed mutual authentication and key exchange scheme between the fog users and servers, without involving the cloud server. Although this scheme uses only hash function and ECC operation, it does not offer perfect forward secrecy feature, and is vulnerable to user traceability, data forgery, and offline password guessing attacks.

Other studies have focused on implementing mutual authentication between IoT devices and fog servers in fog-driven healthcare systems. Jia et al. (2019) designed an authentication and key agreement protocol that uses bilinear pairings to enable mutual authentication between the IoT and fog devices in the healthcare system. This protocol uses the cloud as the third party and shares the generated session key with the IoT, fog, and cloud parties. Although this protocol is immune to man-in-the-middle, message replay, and stolen device attacks, it increases the computational cost and transmission delay, which is unsuitable for limited resource devices and real-time applications. Both Chen et al. (2021) and Wu et al. (2021b) found that Jia's scheme is vulnerable to ESL attack, compromising session key security. They proposed enhanced schemes addressing this issue and ensuring secure communication but also increasing computational demands and communication latency due to reliance on cloud server while establishing the session key.

In the context of Internet of Vehicular (IoV) networks, Ma et al. (2019) proposed three-party authenticated key Agreement scheme between fog user, fog server, and cloud server in a fog-based vehicular IoT network. Their scheme increased the number of scalar point multiplications up to 17 and removed the bilinear pairings, compared with Jia's scheme. However, this scheme doesn't provide the user's identity anonymity and untraceability features and is vulnerable to ESL attack (Zhang et al. (2020)). To overcome these limitations, Eftekhari et al. (2021) proposed an enhanced scheme that reduced the scalar point multiplications to nine. Despite this improvement, their scheme does not offer perfect forward secrecy feature. Wazid et al. (2019a) introduced mutual authentication and key management protocol between vehicles

and fog servers, RSUs (Roadside Units) and fog servers, and between fog and cloud servers in IoV environment. This protocol employs hash functions, bitwise XOR, ECC encryption, and fuzzy extractors. However, it does not ensure confidential communication sessions and is vulnerable to IoT/fog impersonation and known session key attacks. Wu et al. (2021a) also adopted the same cryptographic operations for their scheme, which facilitates mutual authentication and key exchange between vehicles and fog nodes in a VANET (Vehicular Ad-hoc Network). Nevertheless, their approach results in high computational cost and requires the presence of a registration authority for establishing the session key.

Our approach, in contrast, uses only secure hash function and bitwise XOR operation. This strategy ensures lightweight mutual authentication, ideally suited for resource-constrained devices. By avoiding cloud involvement and integrating a trusted node within the fog layer for registration, our method significantly reduces communication latency. It maintains mutual and independent authentication between IoT and fog devices and offers enhanced security and functionality, effectively countering various security threats.

The above-mentioned studies are summarized in Table 1, which outlines the proposed security schemes, the cryptographic techniques they utilize, the methods applied for security verification, and their limitations. In this context, various cryptographic models and algorithms are referenced. *ROM* refers to the Random Oracle Model, *ROR* to the Real-Or-Random model, and *BAN* to the Burrows-Abadi-Needham logic. Additionally, *AES* stands for the Advanced Encryption Standard algorithm, *DES* for the Data Encryption Standard algorithm, *RSA* for the Rivest-Shamir-Adleman encryption algorithm, *HMAC* for Hash-based Message Authentication Code, *DoS* for Denial of Service attack, *MiTM* for Man-in-The-Middle attack, and *RSU* for Road Side Unit.

3. System architecture and threat model

This section presents the system architecture and defines two threat models to evaluate the security of our scheme.

3.1. System architecture

Fig. 1 presents the typical system architecture of fog computing that comprises the cloud, fog, and IoT devices layers. The topmost layer is the cloud, which has many data centers with unlimited computational and storage capacities. However, these data centers are located physically far from the IoT devices. In contrast, the fog layer is located between the cloud layer and IoT devices and includes one central node called the Registration Authority (RA) and many processing fog servers (*F*) with limited resources. The registration authority is a trusted central fog server that registers the IoT and fog devices securely in the fog layer. However, we assume that the registration authority is not faulty and is working in good condition close to the fog servers and connected with them by high-speed links, making the communication delay negligible in the fog layer. In contrast, the registration authority connects with the cloud over the Internet. The lowest layer of the architecture is the IoT devices layer, which involves various IoT devices (*D*) such as vehicles, smartphones, robots, etc. The IoT devices are connected to the fog servers wirelessly, and both the IoT devices and fog servers are considered untrusted parties. Therefore, when the IoT device wants to interact securely with the fog server, they should authenticate each other without resorting to the registration authority.

3.2. Threat model

In this work, we adhere to the Dolev-Yao (DY) threat model (Dolev and Yao (1983)). Based on this model, we have the following assumptions:

- The network links between the IoT devices and fog servers are insecure and public.

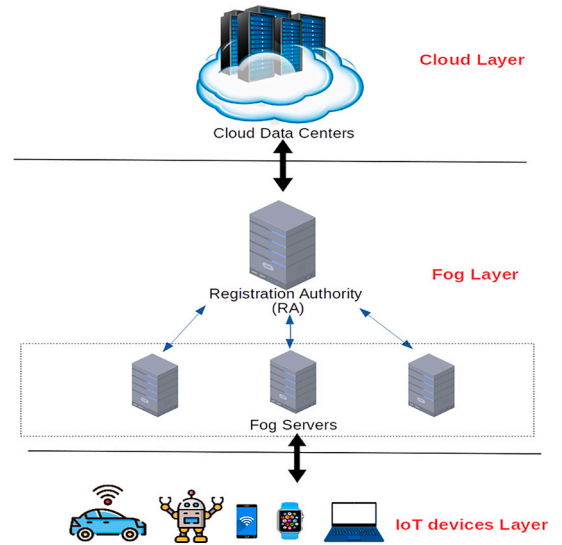


Fig. 1. Fog computing architecture.

- The attacker has full control of the public links between the IoT devices and fog servers, which means that the attacker can replay, delay, eavesdrop, modify, delete, and reorder the transmitted messages over these links.
- The endpoints of the link (i.e., IoT device, fog server) are not trusted parties, while the registration authority is considered a trusted party and can not be penetrated.

We also consider the Canetti-Krawczyk (CK) adversary model (Canetti and Krawczyk (2001)). According to this model, there are the following assumptions:

- The adversary can get the secret keys and other information stored in the captured IoT device or fog server using a power analysis attack.
- The adversary can use the compromised data to apply many attacks, such as session key exposure, man-in-the-middle, identity impersonation, and replay attacks.

4. Proposed scheme

This section presents LAAKA, a Lightweight Anonymous Authentication and Key Agreement scheme in fog-driven IoT systems. This scheme enables IoT devices and fog servers to authenticate each other without the help of the registration authority. The notations adopted in this work and their meaning are illustrated in Table 2. The proposed scheme consists of the three following phases.

4.1. Initialization phase

The registration authority (RA) should do this phase before registering the IoT and fog devices. RA generates a long-term master secret key (*K*) and store it securely in a protected place. It also chooses a secure hash function *h(.)*. We assume that the registration authority is a fully trusted party in the fog layer, which stores all the information of the registered IoT devices and fog servers in a secure database.

4.2. Registration phase

Each IoT device or fog server should register itself with the registration authority when it joins the network as follows:

Table 1
Summary of related work.

Reference	Authentication Scheme	Cryptographic Technique	Security Verification	Limitation (s)
Ali and Sridevi (2022)	Credential-based mutual authentication protocol between IoT devices and fog servers.	Hash function & symmetric encryption (DES) & asymmetric encryption (RSA)	iFogSim	• Authentication requires cooperation from the registration authority. • Vulnerable to device capture attack. • High computational cost.
Ibrahim (2016)	Lightweight mutual authentication protocol for fog users with both existing and newly joined fog servers.	Hash function & asymmetric encryption (RSA)	BAN logic	• Lack of user anonymity. • Vulnerable to IoT device traceability attack. • Not resistant against MiTM attack.
Rahman and Wen (2019)	Anonymous mutual authentication and key agreement between edge devices and fog servers.	HMAC & symmetric encryption (AES)	BAN logic	• Utilizes a long-term master key for session key generation. • Requires the presence of registration authority to establish and distribute the session key. • High communication latency.
Hamada et al. (2022)	Lightweight anonymous mutual authentication and key agreement scheme between fog users and servers, independent of the registration authority	Hash function & XOR, symmetric encryption (AES) & ECC encryption	BAN logic	• No resistance against device capture attack. • High computational cost. • High communication latency.
Wang et al. (2020)	Anonymous mutual authentication protocol between fog gateways and cloud server, with dynamic session key updates among fog gateways.	Hash function & XOR & symmetric encryption (AES)	AVISPA & BAN logic	• Vulnerable to device capture attack. • High computational cost.
Vinoth et al. (2020)	Secure multi-factor authenticated key agreement scheme between the user and sensor in Industrial IoT.	Hash function & XOR & symmetric encryption (AES)	ROR model	• Vulnerable to sensor node capture. • Not resistant against DoS attack. • Not resistant against replay attack.
Amor et al. (2017)	Anonymous mutual authentication protocol for fog users and servers.	Hash function & ECC encryption & bilinear pairing	AVISPA	• High computational cost. • No protection of the session key between the user and server. • Vulnerability to fog server impersonation attack.
Weng et al. (2021)	Lightweight anonymous mutual authentication between the edge user/device and fog server.	Hash function & XOR	BAN logic	• Authentication requires cooperation from the registration authority. • Vulnerable to replay attack. • High communication latency.
Rana et al. (2021a)	Lightweight mutual authentication protocol between the fog user and remote server	Hash function. & XOR	ROM	• Vulnerable to user impersonation. • Vulnerable to ESL attack. • Does not provide user anonymity. • Potential to user traceability.
Guo and Guo (2021)	Anonymous handover authentication and key agreement protocol between the mobile IoT device and fog node.	Hash function& XOR	ROR model	• Vulnerable to DoS attack. • Potential to user traceability. • Lack of shared session key security.
Wazid et al. (2019b)	Secure key management and user authentication scheme (SAKA-FC) in fog computing	Hash function. & XOR & ECC encryption	AVISPA& ROR model	• Prone to clogging attack. • Vulnerable to user impersonation. • Susceptible to DoS attack. • Exposes traceability risk. • Vulnerable to device capture attack.
Ali et al. (2021b)	Improve the security of SAKA-FC scheme proposed by Wazid et al. (2019b).	Hash function & XOR& ECC encryption	AVISPA & BAN logic	• Vulnerable to fog node impersonate attack. • High computational cost.
Patonico et al. (2019)	Identity-based mutual authentication and key agreement protocol between the IoT device and fog server.	Hash function & ECC encryption	ROM	• Vulnerable to device capture/ stolen attack. • Vulnerable to replay attack.
Singh and Chaurasiya (2021)	Anonymous mutual authentication scheme between IoT devices and fog nodes and between fog and cloud servers without key management	Hash function & ECC encryption	AVISPA & BAN logic	• Vulnerable to internal spoofing attack. • Involving the could server in the authentication procedure. • High communication latency.
Guo et al. (2020)	Mutual authentication between remote users and fog servers in smart home environment, excluding centralized cloud servers.	Hash function & XOR & symmetric bivariate polynomial	ROR model	• Vulnerable to ESL attack. • Vulnerable to fog device capture attack.

(continued on next page)

Table 1 (continued)

Reference	Authentication Scheme	Cryptographic Technique	Security Verification	Limitation (s)
Gope (2019)	Three privacy-preserving, lightweight authentication protocols in fog-cloud environment.	Hash function & XOR & symmetric encryption (AES)	BR93- Model Bellare and Rogaway (1993)	• Vulnerable to IoT device capture. • Susceptibility to known key session attack. • Prone to masquerading attack. • Vulnerable to DoS attack. • Lack of Perfect Forward Secrecy.
Kalaria et al. (2021)	Mutual authentication and key exchange between the user and fog server, excluding centralized cloud servers	Hash function & ECC encryption	AVISPA & BAN Logic	• Lack of Perfect Forward Secrecy. • Potential for user traceability. • Risk of data forgery. • Susceptibility to offline password guessing attack.
Jia et al. (2019)	Mutual authentication and key agreement scheme for IoT users and fog servers in healthcare systems.	Hash function & XOR & ECC encryption & bilinear pairing	BRP model Bellare et al. (2000)	• Lacks security of the session key. • Vulnerable to ESL attack.
Chen et al. (2021)	Mutual authentication and key exchange scheme for IoT users and fog servers in healthcare systems.	Hash function & XOR & ECC encryption & bilinear pairing & fuzzy extractor	ProVerif	• Dependent on the registration authority for establishing and distributing the session key. • High communication latency.
Wu et al. (2021b)	Mutual authenticated key agreement protocol for IoT users and fog servers in healthcare systems.	Hash function & XOR& ECC encryption & bilinear pairing	ProVerif	• Dependent on the registration authority for establishing and distributing the session key. • Vulnerable to offline guessing attack. • High computational cost.
Ma et al. (2019)	Three-party authenticated key agreement protocol involving fog users, fog servers, and cloud servers within a fog-based vehicular IoT network.	Hash function & XOR& ECC encryption	BRP model Bellare et al. (2000)	• Lack of user anonymity. • Potential to user traceability. • Vulnerable to ESL attack.
Eftekhari et al. (2021)	Three-party authenticated key agreement protocol involving fog users, fog servers, and cloud servers within a fog-based vehicular IoT network.	Hash function & XOR& ECC encryption	ProVerif	• Absence of Perfect Forward Secrecy feature.
Wazid et al. (2019a)	Mutual authentication and key management protocol between vehicles and fog servers, RSUs and fog servers, fog servers and cloud servers in Internet of Vehicles environment.	Hash function & XOR& ECC encryption & fuzzy extractor	AVISPA & ROR model	• No confidential communication session. • Vulnerable to IoT/ fog impersonation attack. • Vulnerable to known session key attack. • Vulnerable to ESL attack.
Wu et al. (2021a)	Mutual authentication and key-exchange system between vehicles and fog nodes in VANET network.	Hash function & XOR& ECC encryption & fuzzy extractor	ProVerif & BAN logic	• Establish the session key requires the presence of the registration authority. • High computational cost.
LAACA	Lightweight anonymous mutual authentication and key agreement scheme between the IoT device and fog server.	Hash function & XOR	Scyther & BAN logic & ROM	

4.2.1. Registration of fog server

The fog server F_i picks a unique identifier ID_f and a secret random number r_1 , which are hashed to yield output A_f . This output is securely transmitted to the registration authority RA , which then issues a temporary identifier TID_f . This TID_f is securely conveyed to the fog server for storage, while the RA also keeps a secure record of both TID_f and A_f , thereby preserving the system's integrity and confidentiality as shown in Fig. 2.

4.2.2. Registration of IoT device

The IoT device D_j picks a unique identifier ID_d and generates a secret random number r_2 . It then registers with RA by calculating $A_d = h(ID_d \parallel r_2)$ and sending it securely. Upon receiving A_d , RA generates a temporary identifier TID_d and a unique secret value B_k , calculated as $h(A_d \parallel A_f \parallel K)$, storing them all securely. RA then sends (TID_d, TID_f, A_f, B_k) to the IoT device and (TID_d, A_d, B_k) to the associated fog server through secure channels. Both the IoT device and

Table 2

Main notations.

Notation	Definition
RA	Registration Authority
D_j, F_i	j^{th} IoT device and i^{th} fog server
K	Long-term master secret key
SK	Session Key
ID_d, ID_f	Unique Identifier of IoT device, fog server, respectively
TID_d, TID_f	Temporary Identifier of IoT device, fog server, respectively
r_1, r_f	Random numbers generated by the fog server
r_2, r_d	Random numbers generated by the IoT device
T_d, T_f, T_s	Timestamps
$h(\cdot)$	Collision-free hash function
$\oplus$	Bitwise XOR operation
$\parallel$	Concatenation operation

the fog server store the received values in memory as shown in Fig. 3. For each IoT device, RA generates a unique B_k to be shared with its

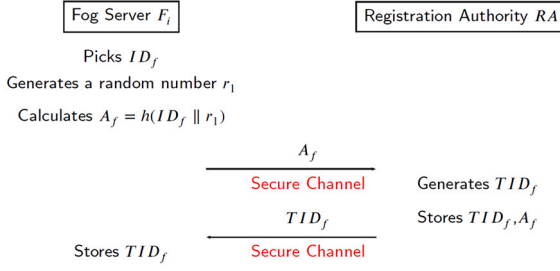


Fig. 2. Registration of fog server.

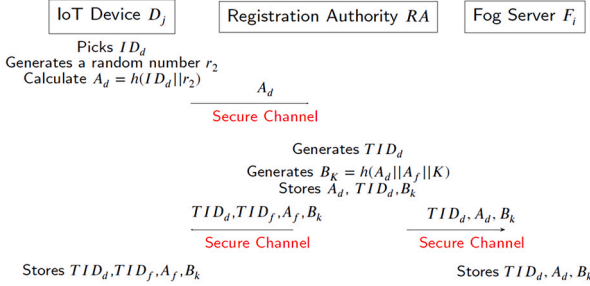


Fig. 3. Registration of IoT device.

associated fog server, reflecting the specific fog area deployment of the IoT device. Notably, the real identifiers of the IoT device and fog server remain unknown to RA, and the device generates a new random number each time it registers with RA, ensuring the unpredictability and uniqueness of the registration credentials.

4.3. Authentication and key agreement phase

We assume that the IoT device D_j is deployed in a specific area of the fog server F_i and is prepared to transmit data to F_i . Before this data submission, D_j and F_i must complete a mutual authentication process, as depicted in Fig. 4. Unlike the scheme proposed by Weng et al. (2021), where the registration authority (RA) in the cloud is involved in both authentication and key generation, our proposed phase is implemented without the assistance of RA. This design emphasizes a more direct connection between the device and the fog server, potentially reducing complexity and dependence on additional entities.

Step 1: D_j generates a timestamp T_d and a unique random number r_d to compute $C_d = h(T_d || r_d)$, ensuring freshness and replay attack prevention. D_j then responds to a challenge A_f from F_i by calculating $E_d = r_d \oplus h(B_k || A_f)$, confirming the response's origin from D_j . To guarantee F_i derives the accurate r_d value, a new temporary identifier is computed as $TID_d^{new} = TID_d \oplus r_d$. Additionally, $G_d = h(A_d || TID_d^{new} || B_k || r_d)$ ensures that F_i discerns the correct TID_d and r_d . By integrating $A_d = h(ID_d || r_2)$ within G_d , F_i can validate D_j 's identity. These components, packaged as $Auth_{Req} = \{TID_d, T_d, C_d, E_d, G_d\}$, are transmitted to F_i over a public channel.

Step 2: Upon receiving the authentication request $Auth_{Req}$ from D_j , the fog server F_i first validates the temporary identifier TID_d by comparing it with the one received earlier from RA. It also verifies the timestamp's authenticity by ensuring that the difference between the current timestamp T_c and the received T_d is within the allowed network latency ΔT . If the message is delayed beyond this limit, F_i recognizes it as a potential replay attack and dismisses it. F_i will proceed with the authentication process only if both TID_d and T_d successfully pass these checks. Otherwise, the request is rejected, and the session is terminated.

Step 3: F_i knows its generated random r_1 and ID_f that are used to calculate A_f . It also knows B_k , since it was received from RA. F_i first extracts the received r_d^* as $r_d^* = E_d \oplus h(B_k || A_f)$. It then scrutinizes r_d to detect any manipulation by calculating $C_d^* = h(T_d || r_d^*)$

and comparing it with the received C_d . Furthermore, F_i calculates $TID_d^{new*} = TID_d \oplus r_d^*$ and $G_d^* = h(A_d || TID_d^{new*} || B_k || r_d^*)$ to verify if G_d^* matches the received G_d . This procedure confirms that F_i is interacting with the legitimate IoT device D_j , guarantees the accurate submission of A_d , and ensures that no illicit device has intruded. Upon successful authentication, F_i proceeds to the next step; otherwise, the session is terminated.

Step 4: F_i begins by generating a timestamp T_f to ensure the freshness of its authentication reply $Auth_{rep}$, and selects a random number r_f . It then computes $C_f = h(T_f || r_f)$. To maintain the session key's freshness, F_i also creates a timestamp T_s and calculates a session key $SK = h(r_d || r_f || T_s)$, which varies with each new session due to its reliance on randomly generated numbers and timestamps.

Step 5: F_i then computes $E_f = r_f \oplus h(TID_d^{new*})$, to affirm to D_j that it has received the correct r_d . Additionally, it calculates $TID_f^{new} = TID_f \oplus r_f$, and $G_f = h(TID_f^{new} || B_k || r_f || SK || T_s)$. These computations are designed to securely transmit the session key SK and thwart impersonation attacks by rendering the random number r_f impervious to prediction. Finally, F_i sends $Auth_{rep} = TID_f, T_f, T_s, C_f, E_f, G_f$ to D_j over a public channel.

Step 6: Once D_j receives $Auth_{rep}$, it verifies if the received TID_f is consistent with what was previously received from RA. It then ensures that this reply has been received within the allowable time frame by comparing it with its current time, according to the condition $T_c - T_f \leq \Delta T$. If both TID_f and T_f are found to be valid, then D_j will proceed with the next phase of the authentication process. Otherwise, the message $Auth_{rep}$ will be rejected, and the session will be terminated.

Step 7: To validate the associated F_i 's knowledge of the exact r_d value, D_j retrieves the TID_d identifier from its memory and calculates $TID_d^{new} = TID_d \oplus r_d$. This value helps D_j to derive the r_f^* value as $r_f^* = E_f \oplus TID_d^{new}$. Next, D_j determines the legitimacy of r_f^* by calculating $C_f^* = h(T_f || r_f^*)$, and comparing it with the received C_f . Additionally, D_j validates T_s , calculates the session key as $SK^* = h(r_d || r_f^* || T_s)$, $TID_f^{new*} = TID_f \oplus r_f^*$, and $G_f^* = h(TID_f^{new*} || B_k || r_f^* || SK^* || T_s)$. If G_f^* matches G_f , D_j confirms that both the correct r_f value and valid session key SK were received, while F_i obtained the correct r_d value. Consequently, D_j recognizes F_i as a legitimate entity, progressing to the next step; otherwise, the session is terminated by D_j .

Step 8: After establishing trust with F_i , D_j proceeds to prove that it possesses the correct session key. To do this, it sends an acknowledgment over a public channel to F_i . The acknowledgment is computed as $Ack = h(r_f^* || B_k || SK^*)$, ensuring that only the party with the correct session key can validate this proof. By providing this acknowledgment, D_j confirms the legitimacy of the session key to F_i .

Step 9: Upon receipt of the acknowledgment (Ack) from D_j , F_i calculates the hash value of $h(r_f || B_k || SK)$ and compares it with the received Ack . A match between these values not only substantiates the authenticity of both the IoT device and the fog server but also verifies that they both possess the same session key. This established key is then utilized to encrypt subsequent communications between them, ensuring a secure connection.

5. Security analysis

This section provides a comprehensive examination of the security measures within our scheme, demonstrating its robust defenses against various cyberattacks. Along with an informal security analysis that illustrates the scheme's resilience, we also explore more formalized methods. By using Burrows-Abadi-Needham (BAN) logic and Random Oracle Model (ROM), we conduct a rigorous security assessment to validate the theoretical soundness of our design. Additionally, we utilize the Scyther tool, a prominent tool in security analysis, to empirically evaluate the robustness of the proposed protocol, further solidifying its reliability and effectiveness in real-world applications.

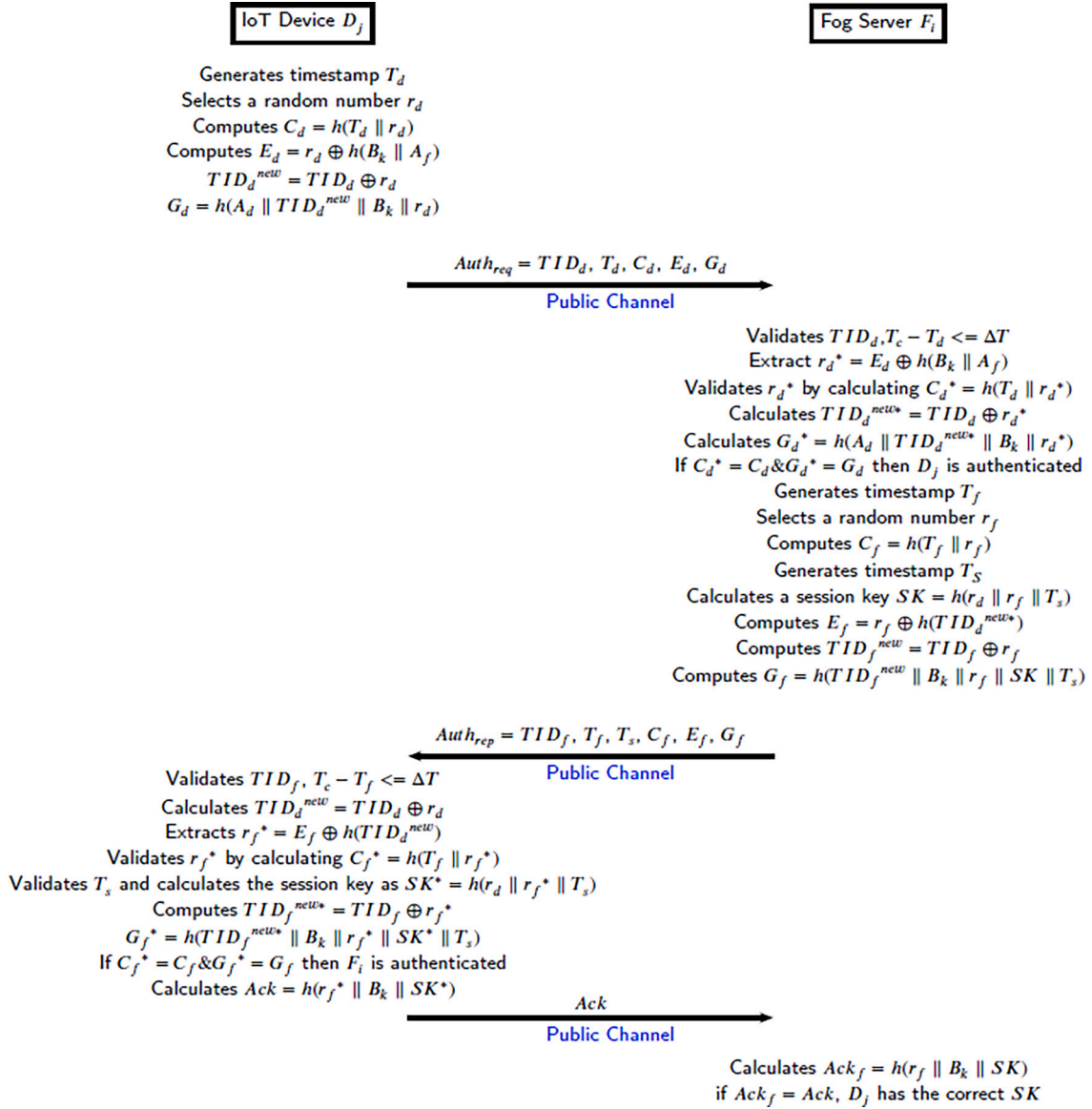


Fig. 4. Mutual authentication and key agreement.

5.1. Informal security analysis

Our scheme (LAAKA) is robust against various cyberattacks, as explained below:

Man-in-The-Middle (MiTM) attack: This type of attack occurs when an unauthorized party intercepts the messages transmitted between the IoT device D_j and fog server F_i . The proposed protocol counters this threat by utilizing hash functions to create secret values (C_d , C_f , G_d , G_f , SK). This construction ensures that reversing the hash to find the original values is computationally intractable. Furthermore, the protocol employs comprehensive authentication checks, including the verification of hash values, temporary identifiers, and timestamps. So, a MiTM attacker needs to forge these values accurately to impersonate one of the parties, which should be difficult without access to the secret values A_d , A_f , B_k . The session key (SK), derived from securely exchanged random values (r_d , r_f), further fortifies the protocol. A MiTM attacker, despite intercepting messages, finds calculating the session key impossible due to the inaccessibility of these random values. Consequently, the integrity and authenticity of the communication remain intact, making the protocol highly resilient to MiTM attacks.

Replay attack: Occurs when an attacker intercepts a valid message and resends it later, intending to deceive the recipient into believing it's legitimate. Our scheme counters this threat through multiple strategies. Firstly, we use timestamps (T_d , T_f , T_s) to ensure messages are time-sensitive; both the IoT device and the fog server check their freshness, dismissing any that fall outside a designated time window. Next, for each session, distinct random values r_d , r_f are generated, adding another layer of complexity for any attacker trying to resend a previous message. The session key (SK) is calculated based on these random values and the timestamp T_s . So, it is a session-specific key and can't be reused. Additionally, the protocol can detect any altered timestamps (T'_d or T'_f). If an attacker generates new random numbers (r_a) and calculates corresponding hash values as $h(T'_d \parallel r_a)$ or $h(T'_f \parallel r_a)$, it still cannot compute G_d , and G_f without accessing the secret values A_d , A_f , and B_k . These combined measures create a robust defense against replay attacks.

Ephemeral-Secret-Leakage (ESL) attack: Aims to reveal temporary secrets used during the authentication and establish the session key. In this protocol, the ephemeral secrets that might be targeted in an ESL attack would include the random numbers (r_d , r_f) and potentially the session keys derived from them. These random numbers are encrypted

using a bitwise XOR operation with hash values. These hash values include secret parameters like (A_d, A_f, B_k) , making it challenging for an attacker to extract the original random numbers without knowing these values. These random numbers are used in hash functions with other parameters and sent over the public channel. Since the hash function is considered one-way, extracting the original values should be computationally infeasible for an attacker. The session key SK is derived using these random numbers along with a timestamp T_s and hashed with other values before submitting, making the exposure of the session key unlikely.

Stolen Verifier attack: Occurs when an attacker obtains the verification data used during authentication. In the context of our protocol, the threat is mitigated by not storing the real identifiers (ID_d, ID_f) , which are essential for generating A_d or A_f . Consequently, even if an attacker accesses the stored verification information, it will not have enough details to impersonate a legitimate device. The protocol's incorporation of secret random numbers (r_1, r_2, r_d, r_f) that remain unknown to the attacker adds to the complexity, as these numbers are required to produce accurate authentication values. The attacker's inability to generate the correct A_d and the subsequent G_d , both dependent on the real identifier and the random numbers, will result in authentication failure. Without the correct random values, the attacker also can't compute the proper session key, further hindering their impersonation attempts.

Key Exposure attack: In our proposal, the fog server generates a unique session key (SK) for each connection with an IoT device. This specificity ensures that the security of one session doesn't compromise others. The session key is exclusively known to the fog server and the associated IoT device. Other parties, such as the Registration Authority (RA), do not have access to this secret key. This restriction further isolates the session, ensuring that even if other parts of the system are compromised, the session key remains secure. The attacker's inability to access the randomly generated numbers by the IoT device and the fog server during the authentication process prevents them from reverse-engineering or otherwise exposing the session key.

Known Session Key attack: The adversary may obtain an old known session key to access the secret parameters and session keys of the subsequent communication sessions. According to our proposed approach, the session key is a hashed value of random numbers. It is computationally infeasible to know these numbers by obtaining the hash value because of the one-way characteristic of the hash function. However, the adversary is unable to get the current generated random numbers to deduce a new session key by knowing of previously compromised keys. Consequently, our scheme is immune to the known session key attack.

Unknown Key-Share attack: During the registration phase, the Registration Authority (RA) securely transmits TID_d, TID_f, A_f to the IoT device and TID_d, TID_f, A_d to the fog server, thereby enabling them to recognize each other. If an unauthorized third entity (with identifiers ID_a and TID_a) attempts to create a session key with the fog server, it will be unable to do so. The failure arises because the unauthorized entity cannot produce a value for G_a that matches G_d , due to a lack of knowledge about A_d , causing a validation failure at the fog server. Similarly, if this party tries to create a session key with the IoT device, it also fails. Without knowledge of the random value r_d , it can't extract r_f , causing validation of TID_a with TID_f to fail at the IoT device side. Thus, our scheme thwarts the attacker's efforts to trick the IoT device and fog server into establishing a shared secret key with it, thereby reinforcing the protocol's immunity to unknown key-share attacks.

IoT device impersonation attack: An attacker attempting to pose as a legal IoT device to create a session key with the fog server will face insurmountable barriers due to the unknown secret parameters B_k and A_d , as well as the actual random numbers (r_2, r_d) generated by the IoT device. Without knowledge of the real identifier of the IoT device ID_d and its generated random number r_2 , the attacker is unable to calculate the correct G_d , thus rendering impersonation infeasible, even if

E_d, G_d are publicly transmitted. Adding to this, the fog server issues a challenge A_f through secure channels known only to the trusted RA and IoT device. This challenge is hashed with the secret value B_k , and combined with the unique random number r_d for each session, further complicating the attacker's ability to impersonate. Moreover, the session validation process rigorously compares various calculated and received values such as C_d^* and C_d , G_d^* and G_d , ensuring that even if an attacker intercepts the authentication request, they cannot extract the random number r_f from the fog server due to the unknown random number r_d , nor validate G_f without knowledge of B_k and A_f . This multi-layered approach guarantees robust protection against IoT device impersonation.

Fog server impersonation attack: The attacker tries to present as a legal fog server to create a session key and share it with the IoT device. However, it lacks knowledge of the random numbers r_d and r_f , as well as the secret parameters B_k and A_f shared between the legitimate fog server and the IoT device. Thus, the attacker is unable to calculate the correct value of G_f , making it detectable by the IoT device. The attacker's lack of access to B_k and A_f will also prevent the correct calculation of E_f , which relies on the XOR operation with these values. The IoT device, knowing the legitimate values, can recognize the incorrect E_f coming from an impersonator. A discrepancy in any of these values would lead to the termination of the session, preventing the impersonation attack from progressing.

Physical Capture/Stolen/Lost of IoT device attack: In the event of an IoT device being physically captured, lost, or stolen, the protocol still protects against this type of attack due to its secure design. An attacker (even with access to stored parameters (TID_d, TID_f, A_f, B_k) and the ability to eavesdrop on exchanged messages) can't know the real identifier of the IoT device (ID_d) and its generated random numbers r_2 and r_d . This lack of information prevents the attacker from generating the correct A_d , thereby making it impossible to produce the correct G_d value. The fog server's rigorous validation checks (including TID_d, T_d, C_d, E_d , and G_d) ensure that any incorrect information leads to the rejection of the authentication request. The protocol's unpredictability and uniqueness, achieved through the generation of new random numbers each time the device registers, further prevent impersonation attempts. Without knowing the correct A_d value, the attacker cannot ascertain the long master key K , which is concatenated with A_d and A_f to generate B_k . Also, the attacker's ignorance about the generated random number r_d means it cannot derive the current session key shared with the fog server. By generating distinct random numbers for different sessions, the system ensures that compromising one IoT device doesn't lead to compromising others connected with the fog server, thereby resisting capture/stolen/loss of IoT device attacks.

Physical Capture of Fog server attack: If a fog server storing parameters TID_d, TID_f, A_d , and B_k , is captured by an attacker, the attacker can access these stored secret values and eavesdrop on exchanged messages. However, the protocol's design ensures that the attacker still lacks knowledge of the random number r_1 generated by the fog server and, consequently, the value of A_f . This makes the attacker incapable of determining the value of r_d , defined as $r_d = E_d \oplus h(B_k \parallel A_f)$. As a result, even if the attacker captures the fog server, it remains unable to generate a session key and share it with the IoT device. Furthermore, the threat does not spread to other servers, as each fog server generates distinct keys for its associated IoT devices, ensuring that compromising one fog server will not lead to compromising others.

Traceability of IoT device attack: The attacker may aim to know the physical location of the IoT device by monitoring the transmitted messages. Using identical parameters for each communication with the fog server makes the IoT device traceable by the attacker. However, our scheme mitigates this risk by requiring both the IoT device and fog server to generate unique hashed values A_d and A_f for each registration with the Registration Authority (RA). The RA then creates different temporal identifiers TID_d, TID_f for each communication session, ef-

effectively thwarting the ability of an attacker to trace the location of the IoT device.

Anonymity attack: The real identities of the parties remain anonymous, even if an attacker intercepts all messages transmitted over public channels. This is ensured through the embedding of the real identifiers (ID_d, ID_f) in hash functions with the random numbers (r_1, r_2), which remain inaccessible to the attacker. The robustness of this security feature is further highlighted by the fact that even the Registration Authority (RA) cannot discern the real identity of the parties, effectively thwarting an attacker's ability to compromise the anonymity of the involved IoT device and fog server.

Denial of Service (DoS) attack: The proposed protocol is designed to be lightweight and resistant against DoS attacks. Unlike many cryptographic schemes that rely on computationally heavy operations such as scalar multiplication or asymmetric encryption, this protocol primarily utilizes hash functions and XOR operations, which are generally less resource-intensive. This design choice makes it particularly suitable for IoT and fog devices with limited computational capabilities. By rejecting any duplicated requests with the same nonce and timestamp, the fog server ensures that the protocol is resilient against replay attacks, further enhancing its defense against DoS. Message validation at each step, including the comparison of timestamp and hash values, adds robustness by terminating any sessions that present inconsistencies, thus preventing attackers from overloading the system. Finally, mutual authentication between the IoT device and the fog server safeguards against unauthorized communication, providing a comprehensive defense strategy. Thus, all these measures contribute to resisting DoS attacks.

Assuming that the adversary can obtain B_k , it can't get any useful information through B_k , because B_k is always contained by the hash function and the adversary has no access to other information.

However, the session key SK is calculated as $SK = h(r_d \parallel r_f \parallel T_s)$, where r_d, r_f are unique random numbers generated by the IoT device, and fog server, respectively for each session. The adversary can't obtain these random numbers. Moreover, assuming that the adversary can access the messages transmitted over the public channels. Although knowing B_k by the adversary, it still can't get the correct hash values of $C_d, E_d, G_d, C_f, E_f, G_f$ due to unknowing the distinct generated random numbers r_d, r_f . Therefore, our scheme provides PFS security feature.

Ensuring Perfect Forward Secrecy (PFS): As described by Günther (1990), PFS requires that leakage of the long-term secret key used to generate the session key will not cause the leakage of the session key used in previous communication. In our scheme, the registration authority (RA) creates and securely stores a long-term master secret key (K). For each pair of IoT devices and fog servers, RA computes a secret value B_k , calculated as $B_k = h(A_d \parallel A_f \parallel K)$. Even if the adversary can obtain B_k , it will not get any significant information through it. This is because B_k is always contained by the hash function, and the adversary lacks access to other critical information such as the random numbers r_1, r_2 and real identifier ID_d, ID_f of the IoT device and fog server. However, the session key SK is calculated as $SK = h(r_d \parallel r_f \parallel T_s)$, where r_d, r_f represent unique, randomly generated numbers by the IoT device and fog server, respectively, for each session. These random numbers remain unknown to the adversary. Therefore, even if the adversary could monitor the public channel communications, knowledge of B_k does not enable it to deduce the correct hash values of $C_d, E_d, G_d, C_f, E_f, G_f$, as it remains unaware of the specific random numbers r_d, r_f used. Hence, our scheme robustly provides the PFS security feature.

Privileged-insider attack: Assuming the adversary is a privileged insider within the RA server, with access to the RA's secure storage (where the master secret key K , temporary identifiers ID_d, TID_f , and the secret values A_d, A_f are stored). Despite possessing this information, the adversary cannot successfully obtain the session key. This is due to their inability to predict the random numbers r_d, r_f which are securely encapsulated within hash values and exclusively shared between the IoT device and fog server. Owing to the inherent security

properties of the hash functions, it becomes computationally infeasible to accurately guess the random numbers, which are critical for deriving the correct transmitted values such as $C_d, E_d, G_d, C_f, E_f, G_f$. Therefore, we can say that our scheme is resistant against privileged-insider attack.

5.2. Security proof using BAN logic

We adopt BAN logic (Burrows et al. (1989)) to demonstrate our scheme security and to show how it achieves the mutual authentication goals. We use the symbols A , and B to indicate the involved parties in the authentication protocol, while K represents the secret session key, and both S , and Y represent the statement that builds up the exchanged messages such as random number, timestamp, etc Sowjanya et al. (2020). The BAN logic notations which are used here are:

- $A \models S$: A believes that S is true.
- $A \triangleleft S$: A sees S , A can read and repeat a received message contains S .
- $A \Longrightarrow S$: A has control over S , A is authoritative about S , and it should be trusted based on that.
- $A \sim S$: A once said S , i.e., A sent a message contains S at some time.
- $\langle S \rangle_Y$: S is combined with Y , Y is secret, and it confirms the identity of whoever said S .
- $\#(S)$: The statement S is fresh, S has not been transmitted before the current protocol execution.
- $A \stackrel{K}{\longleftrightarrow} B$, A and B share and utilize a secret key K for communication.

The BAN logic rules are:

- **Message-meaning rule:** $\frac{A \models A \stackrel{K}{\longleftrightarrow} B, A \triangleleft \langle S \rangle_B}{A \models B \sim S}$
- **Nonce verification rule:** $\frac{A \models \#(S), A \triangleleft B \sim S}{A \models B \models S}$
- **Jurisdiction rule:** $\frac{A \models (B \Longrightarrow S), A \models B \models S}{A \models S}$
- **Belief rule:** $\frac{A \models B \models (S, Y), A \models S, A \models Y}{A \models (S, Y)}$
- **Freshness-conjunction rule:** $\frac{A \models \#(S)}{A \models \#(S, Y)}$

To prove our scheme, there are some assumptions:

- **A1:** $D_j \models \#(r_d)$
- **A2:** $F_i \models \#(r_f)$
- **A3:** $D_j \models \#(T_f)$
- **A4:** $F_i \models \#(T_d)$
- **A5:** $D_j \models F_i \Longrightarrow TID_f$
- **A6:** $F_i \models D_j \Longrightarrow TID_d$
- **A7:** $D_j \models D_j \stackrel{B_k}{\longleftrightarrow} F_i$
- **A8:** $F_i \models F_i \stackrel{B_k}{\longleftrightarrow} D_j$
- **A9:** $F_i \models D_j \stackrel{SK}{\longleftrightarrow} F_i, \#(D_j \stackrel{SK}{\longleftrightarrow} F_i)$
- **A10:** $D_j \models F_i \Longrightarrow D_j \stackrel{SK}{\longleftrightarrow} F_i, \#(D_j \stackrel{SK}{\longleftrightarrow} F_i)$
- **A11:** $F_i \models D_j \stackrel{A_f}{\longleftrightarrow} F_i$
- **A12:** $D_j \models D_j \stackrel{A_d}{\longleftrightarrow} F_i$

The goals of our scheme are:

- **G1:** $F_i \models D_j \models r_d$
- **G2:** $F_i \models D_j \models TID_d$
- **G3:** $F_i \models TID_d$

- **G4:** $D_j \models F_i \models r_f$
- **G5:** $D_j \models F_i \models TID_f$
- **G6:** $D_j \models TID_f$
- **G7:** $D_j \models F_i \models D_j \xleftrightarrow{SK} F_i$
- **G8:** $D_j \models D_j \xleftrightarrow{SK} F_i$
- **G9:** $F_i \models D_j \models D_j \xleftrightarrow{SK} F_i$

The ideal forms of the proposed scheme messages are:

- **Msg1:** $\langle r_d \rangle_{K,r_1,r_2}, h(T_d, r_d), h(TID_d^{new}, B_k, r_d)$
- **Msg2:** $\langle r_f \rangle_{r_d}, h(T_f, r_f), h(TID_f^{new}, r_f, B_k, SK)$
- **Msg3:** $h(r_f, B_k, SK)$

Where *Msg1* represents the authentication request *Auth_{req}*, *Msg2* represents the authentication reply *Auth_{req}*, and *Msg3* represents the acknowledgment *Ack*.

According to the defined assumptions, rules, and goals, the analysis steps of our scheme using BAN logic are presented as follows:

H1: There are three secret variables A_d, A_f, B_k known by the fog server and IoT device. So, considering **Msg1**, **A8**, **A11**, **A12**, and **Message-meaning rule**, F_i believes that D_j has sent *Msg1*:

$$\frac{F_i \models D_j \xleftrightarrow{A_d, A_f, B_k} F_i, F_i \triangleleft \langle Msg1 \rangle_{A_d, A_f, B_k}}{F_i \models D_j \models Msg1}$$

H2: This step ensures the freshness of the timestamp (T_d) and the random number (r_d). In accordance with **A4**, **Freshness-conjunction rule** and because of $h(T_d, r_d)$ is one-way hash function, we get:

$$\frac{\#T_d}{\#h(T_d, r_d)} \quad \#r_d$$

H3: In accordance with **H2**, **A4**, and **Freshness-conjunction rule**, F_i believes *Msg1* is fresh, and we get:

$$F_i \models \#Msg1$$

H4: By checking the freshness of *Msg1* and believing that D_j has sent *Msg1*, F_i believes that D_j has knowledge of *Msg1* which includes r_d . In accordance with **H1**, **H3**, and **Nonce verification rule**, we get:

$$\frac{F_i \models \#(Msg1), F_i \models D_j \models Msg1}{F_i \models D_j \models Msg1}$$

H5: F_i believes that D_j knows r_d (as it's part of *Msg1*), satisfying the goal **G1**. In accordance with **H4**, and **Belief rule**, we get:

$$\frac{F_i \models D_j \models Msg1}{F_i \models D_j \models r_d}$$

H6: Once F_i believes that D_j possesses *Msg1* that includes *TID_d*, it also believes that D_j knows *TID_d*, satisfying **G2**. In accordance with **H4**, and **Belief rule**, we get:

$$\frac{F_i \models D_j \models Msg1}{F_i \models D_j \models TID_d}$$

H7: If F_i believes that D_j has authority over *TID_d*, and if F_i also believes that D_j knows *TID_d*, then F_i can believe *TID_d* as well. In accordance with **H6**, **A6**, and **Jurisdiction rule**, we get:

$$\frac{F_i \models (D_j \models TID_d), F_i \models D_j \models TID_d}{F_i \models TID_d}$$

Therefore, **G3** is achieved, and the fog server authenticates the IoT device.

H8: If D_j believes that it shares a secret value (B_k) with F_i , and if D_j

sees a message (*Msg2*) encrypted with B_k , then D_j believes that F_i has sent message *Msg2*. According to **Msg2**, **A7**, and **Message-meaning rule**, we obtain:

$$\frac{D_j \models D_j \xleftrightarrow{B_k} F_i, D_j \triangleleft \langle Msg2 \rangle_{B_k}}{D_j \models F_i \models Msg2}$$

H9: If the timestamp T_f is fresh and it is hashed with r_f , it means that r_f is also fresh. According to **A3**, **Freshness-conjunction rule**, we obtain:

$$\frac{\#T_f}{\#h(T_f, r_f)} \quad \#r_f$$

H10: If r_f is fresh and it is part from *Msg2*, then *Msg2* is also fresh. In accordance with **H9**, **A3**, and **Freshness-conjunction rule**, we obtain:

$$D_j \models \#Msg2$$

H11: If D_j believes a message (*Msg2*) is fresh and F_i has sent this message, then D_j can believe that F_i indeed knows *Msg2*. In accordance with **H8**, **H10**, and **Nonce verification rule**, we obtain:

$$\frac{D_j \models \#(Msg2), D_j \models F_i \models Msg2}{D_j \models F_i \models Msg2}$$

H12: If D_j believes that F_i knows a message (*Msg2*), then D_j can believe that F_i knows the elements of *Msg2*. In this case, D_j believes that F_i knows r_f which satisfies **G4**. In accordance with **H11**, and **Belief rule**, we get:

$$\frac{D_j \models F_i \models Msg2}{D_j \models F_i \models r_f}$$

H13: *TID_f* is also part from the *Msg2*. So, D_j believes that F_i knows *TID_f* that satisfies **G5**. In accordance with **H11**, and **Belief rule**, we get:

$$\frac{D_j \models F_i \models Msg2}{D_j \models F_i \models TID_f}$$

H14: If D_j believes that F_i has authority over *TID_f*, and if D_j also believes that F_i knows *TID_f*, then D_j can believe *TID_f* as well. In accordance with **H13**, **A5**, and **Jurisdiction rule**, we get:

$$\frac{D_j \models (F_i \models TID_f), D_j \models F_i \models TID_f}{D_j \models TID_f}$$

Therefore, **G6** is achieved and the IoT device authenticated the fog server.

H15: If D_j believes that F_i has knowledge of the *Msg2* which also includes the secret session key *SK*, then D_j believes that F_i knows *SK* that is shared between them, satisfying **G7**. In accordance with **H11**, and **Belief rule**, we obtain:

$$D_j \models F_i \models D_j \xleftrightarrow{SK} F_i$$

H16: If D_j believes that F_i has authority over the shared session key (*SK*), and it knows *SK*, then D_j believes that (*SK*) does exist between itself and F_i . In accordance with **H15**, **A10**, and **Jurisdiction rule**, we get:

$$\frac{D_j \models F_i \models D_j \xleftrightarrow{SK} F_i, D_j \models F_i \models D_j \xleftrightarrow{SK} F_i}{D_j \models D_j \xleftrightarrow{SK} F_i}$$

Therefore, **G8** is accomplished.

H17: If F_i believes that there are secret values B_k, SK shared with D_j , and it receives encrypted message (*Msg3*) by them, it means that this message comes from D_j . In accordance with **Msg3**, **A8**, **A9**, and **Message-meaning rule**, we get:

$$\frac{F_i \models D_j \xleftrightarrow{B_k, SK} F_i, F_s \triangleleft \langle Msg3 \rangle_{B_k, SK}}{F_i \models D_j \sim Msg3}$$

H18: Both r_f and SK are proved as fresh values, so the message ($Msg3$) that includes then is also fresh. In accordance with **A2**, **A9**, and **Freshness-conjunction rule**, we obtain:

$$D_j \models \#Msg3$$

H19: If F_i believes a message ($Msg3$) and D_j has sent this message, then F_i can believe that D_j indeed knows $Msg3$. In accordance with **H17**, **H18**, and **Nonce verification rule**, we get:

$$\frac{F_i \models \#(Msg3), F_i \models D_j \sim Msg3}{F_i \models D_j \models Msg3}$$

H20: F_i , after verifying the freshness and authenticity of the message ($Msg3$), acknowledges that D_j holds the correct established session key (SK). In accordance with **H19**, and **Belief rule**, we obtain:

$$F_i \models D_j \models D_j \xleftrightarrow{SK} F_i$$

Thus, **G9** is achieved, and the fog server believes that the corresponding authenticated IoT devices have the correct established session key.

Considering Goals 1–9, we ensure that our scheme enables the IoT device to mutually authenticate with the fog server in addition to creating a session key to maintain the transmissions confidential between them.

5.3. Security proof using the random oracle model

To further prove the robustness of our proposed scheme, we introduce a formal security analysis using the Random Oracle Model (ROM). Recognized as a crucial method for cryptographic verification, the ROM model plays an essential role in demonstrating the resilience of cryptographic protocols against sophisticated adversarial strategies (Koblitz and Menezes (2015)), especially in the fields of the Internet of Things (IoT) and fog computing.

To prove that our scheme is secure under ROM, we must prove that any adversary, particularly one that is bound by probabilistic polynomial-time (PPT) constraints, cannot compromise our scheme without solving the underlying hard problem, given that the hash functions are assumed to behave like ideal random oracles. This implies that for each unique input, $h(\cdot)$ returns a random output from its output space, and this output is consistent for repeated queries with the same input. Under ROM, we assume that $h(\cdot)$ is idealized, providing collision-resistant and preimage-resistant properties that are crucial for our security analysis. Prior to proving the session key security of our scheme in Theorem 1, we briefly discuss the ROM model.

Definition. Consider a hash function h defined over binary strings $\{0,1\}^*$ to $\{0,1\}^w$, representing a deterministic algorithm that produces a fixed-size w -bit binary string, termed as the hash value or message digest, for any input binary string of arbitrary length. The likelihood of an adversary $\mathcal{A}$ discovering two distinct inputs that result in identical hash outputs within a time frame t is expressed as $\text{Adv}_{\mathcal{A}}^{\text{HASH}}(t) = \Pr[(x_1, x_2) \xleftarrow{R} \mathcal{A} : x_1 \neq x_2 \wedge h(x_1) = h(x_2)]$. $\mathcal{A}$: indicates that the adversary picked the input pairs (x_1, x_2) randomly.

Participants. Let $\Pi_{D_j}^a$ and $\Pi_{F_i}^b$ denote the instance a, b of the IoT device D_j and fog server F_i , respectively.

Freshness. If the adversary $\mathcal{A}$ can't get the session key SK between $\Pi_{D_j}^a$ and $\Pi_{F_i}^b$ through applying the *Reveal* query, then they are fresh.

Adversary's Model: Adversary $\mathcal{A}$ can eavesdrop, modify, and replay the transmitted information over the communication link between $\Pi_{D_j}^a$ and $\Pi_{F_i}^b$ via the following oracle queries:

- **Execute** ($\Pi_{D_j}^a, \Pi_{F_i}^b$): Represents a passive attack where $\mathcal{A}$ eavesdrops the communication between $\Pi_{D_j}^a$ and $\Pi_{F_i}^b$.
- **Send** ($\Pi_{D_j}^a, \Pi_{F_i}^b$): Represents an active attack, allowing $\mathcal{A}$ to create, modify, intercept and resend intercept to $\Pi_{D_j}^a$ or $\Pi_{F_i}^b$.
- **Reveal** ($\Pi_{F_i}^b$): The session key SK that is generated by $\Pi_{F_i}^b$ can be uncovered to the adversary $\mathcal{A}$ while executing this query.
- **CorruptIoT** ($\Pi_{D_j}^a$): This query simulates a compromise of D_j , revealing the stored data in $\Pi_{D_j}^a$.
- **CorruptFog** ($\Pi_{F_i}^b$): This query simulates a compromise of F_i , revealing the stored data in $\Pi_{F_i}^b$.
- **Test** ($\Pi_{F_i}^b$): Involves flipping a coin c to decide whether $\Pi_{F_i}^b$ will return the actual session key SK or a random value, based on the coin's outcome, kept secret from $\mathcal{A}$.

Semantic Security of the Session Key. The indistinguishability of our protocol ensures that the session key (SK) generation process is sufficiently random to mitigate the risk of informational disclosure. The adversary $\mathcal{A}$ can pose a series of *test* queries to either $\Pi_{D_j}^a$ or $\Pi_{F_i}^b$. $\mathcal{A}$ is challenged to identify the value of the flipped result c (i.e., the real SK). As the process concludes, the adversary profits a predicted bit, labeled as c' . A successful match is declared if c' is equal to the flipped result c . Let *succ* represent the scenario where $\mathcal{A}$ wins this challenge, and P is our protocol. Since there are only two possible values for c , namely 0 or 1, $\mathcal{A}$ can win this game with a probability of $\frac{1}{2}$ by random guessing (Bellare and Rogaway (2005)). Therefore, the advantage of $\mathcal{A}$, or the excess probability over $\frac{1}{2}$ in breaking the semantic security of protocol P , is defined as:

$$\text{Adv}_P(\mathcal{A}) = |2 \cdot \Pr[\text{Succ}] - 1| \quad (1)$$

If $\text{Adv}_P^{\text{AKE}}$ is negligible, then our scheme is considered secure under the random oracle model.

Theorem 1. Consider the random oracle model where an adversary, denoted as $\mathcal{A}$, operates within a polynomial time frame t to challenge our scheme. Define q_h as the total number of hash queries, q_s as the number of send queries, and q_{ex} as the number of execute queries made by $\mathcal{A}$, the advantage of $\mathcal{A}$ in compromising the security of our P is given as:

$$\text{Adv}_P^{\text{AKE}} \leq \frac{q_h^2}{2^{|Hash|}} + \frac{(q_s + q_{ex})^2}{2^{|Rand|}} + \frac{q_s}{2^{|Rand|-2}} \quad (2)$$

where $2^{|Hash|}$ is the range space of the hash function, while $2^{|Rand|}$ represents the range space of the random number.

We establish a sequence of games $G_i (i = 0, 1, 2, 4)$. The adversary in each game tries to guess the value of flipped result c . Let $\Pr[\text{Succ}_i]$ represent the probability of guessing the correct value of c by $\mathcal{A}$ during the game i .

- **Game (G0):** $\mathcal{A}$ performs a real attack on P at the start of the game, where the value of c is randomly selected. Therefore:

$$\text{Adv}_P^{\text{AKE}} = |2 \cdot \Pr[\text{Succ}_0] - 1| \quad (3)$$

- **Game (G1):** The adversary, denoted as $\mathcal{A}$, aims to increase its chance of correctly guessing the flipped value c . To achieve this, $\mathcal{A}$ runs *Execute* queries to intercept communications between the $\Pi_{D_j}^a$ and $\Pi_{F_i}^b$. Subsequently, $\mathcal{A}$ performs *Test* ($\Pi_{F_i}^b$) query to identify the value of c . In our protocol, the secret key SK is computed by hashing random numbers r_d and r_f . For $\mathcal{A}$ to determine the correct value of SK , it must gain access to r_d and r_f . However, these random values are not transmitted, and uncovering them requires knowing the values of A_d , A_f , and B_k . The computation

of B_k relies on the values of both A_d and A_f , which consist of random numbers and real identifiers, all hashed and unknown. Consequently, $\mathcal{A}$ cannot derive these secret values simply through eavesdropping on C_d , E_d , G_d , C_f , E_f , and G_f . Therefore, this type of eavesdropping can't improve $\mathcal{A}$'s probability of success, making Games G1 and G0 indistinguishable:

$$\Pr[\text{Succ}_1] = \Pr[\text{Succ}_0] \quad (4)$$

- **Game (G2):** Based on G1, $\mathcal{A}$ can send hash queries, and *Send* ($\Pi_{D_j}^a, \Pi_{F_i}^b$) to create, modify, and reply the transmitted messages between $\Pi_{D_j}^a$ and $\Pi_{F_i}^b$. In our proposed protocol, if $\mathcal{A}$ wants to target one of the participants by sending a forged message, it needs to access the secret values r_d, r_f , and B_k to generate the authentic message. These values are embedded in the values of hash. Moreover, all of the transmitted parameters C_d , E_d , G_d , C_f , E_f , and G_f are timestamp dependent and consists of random numbers. Therefore, no collision will occur in hash outputs. According to the birthday paradox, the output hash value might collide with a probability $\frac{q_h^2}{2^{|Hash|+1}}$. All the random numbers used in our protocol are uniformly randomized and the probability of collision for random numbers is $\frac{(q_s + q_{ex})^2}{2^{|Rand|+1}}$. Because $\mathcal{A}$ can't find any collision between any two messages due to containing timestamps and random values in these messages. Thus, we have:

$$|\Pr[\text{Succ}_2] - \Pr[\text{Succ}_1]| \leq \frac{q_h^2}{2^{|Hash|+1}} + \frac{(q_s + q_{ex})^2}{2^{|Rand|+1}} \quad (5)$$

- **Game (G3):** In this game, $\mathcal{A}$ applies *CorruptIoT* ($\Pi_{D_j}^a$) to access the stored information (TID_d , TID_f , A_f , B_k) in the IoT device d . In our protocol, $\mathcal{A}$ can't know the real identifier of the IoT device (ID_d) and its generated random numbers r_2 and r_d . This lack of information prevents the adversary from reconstructing the session key SK . The probability of random numbers is $\frac{q_s}{2^{|Rand|}}$. Hence, we have:

$$|\Pr[\text{Succ}_3] - \Pr[\text{Succ}_2]| \leq \frac{q_s}{2^{|Rand|}} \quad (6)$$

- **Game (G4):** In this game, $\mathcal{A}$ applies *CorruptFog* ($\Pi_{F_i}^b$) to access the stored information (TID_d , TID_f , A_d , B_k) in the fog server f . In our protocol, $\mathcal{A}$ can't know the real identifier of the fog server (ID_f) and its generated random numbers r_1 and r_f . This lack of information prevents the adversary from reconstructing the session key SK . The probability of random numbers is $\frac{q_s}{2^{|Rand|}}$. Hence, we have:

$$|\Pr[\text{Succ}_4] - \Pr[\text{Succ}_3]| \leq \frac{q_s}{2^{|Rand|}} \quad (7)$$

Finally, $\mathcal{A}$ can't guess the correct flipped value c as both D_j and F_i generate independent and random session keys SK . Therefore:

$$\Pr[\text{Succ}_4] = \frac{1}{2} \quad (8)$$

$$\Pr[\text{Succ}_3] \leq \frac{q_s}{2^{|Rand|}} + \frac{1}{2} \quad (9)$$

$$\Pr[\text{Succ}_2] \leq \frac{q_s}{2^{|Rand|}} + \frac{q_s}{2^{|Rand|}} + \frac{1}{2} \quad (10)$$

So,

$$\Pr[\text{Succ}_2] \leq \frac{q_s}{2^{|Rand|-1}} + \frac{1}{2} \quad (11)$$

By combining equation (11) and (5), we get:

$$\Pr[\text{Succ}_1] \leq \frac{q_h^2}{2^{|Hash|+1}} + \frac{(q_s + q_{ex})^2}{2^{|Rand|+1}} + \frac{q_s}{2^{|Rand|-1}} + \frac{1}{2} \quad (12)$$

By combining equation (12), (4) and (3), we get:

$$\text{Adv}_P(\mathcal{A}) \leq \left| 2 \cdot \left(\frac{q_h^2}{2^{|Hash|+1}} + \frac{(q_s + q_{ex})^2}{2^{|Rand|+1}} + \frac{q_s}{2^{|Rand|-1}} + \frac{1}{2} \right) - 1 \right| \quad (13)$$

So,

$$\text{Adv}_P(\mathcal{A}) \leq \left| \frac{q_h^2}{2^{|Hash|}} + \frac{(q_s + q_{ex})^2}{2^{|Rand|}} + \frac{q_s}{2^{|Rand|-2}} \right| \quad (14)$$

5.4. Security proof using Scyther tool

Scyther is an efficient tool used to identify potential attacks and vulnerabilities in security protocols by automatically analyzing the protocols and scrutinizing their behavior (Cremers (2008)). It offers more security features than other tools and verifies the security properties of the protocol against a variety of attacks. Scyther operates based on a pattern refinement algorithm, providing concise representations of (infinite) sets of protocol cases. This functionality enables the tool to evaluate various types of attacks and potential protocol behaviors. The security protocols are verified against threats such as Dolev-Yao adversary (Dolev and Yao (1983)) and internal attacks (Basin and Cremers (2014)). This tool has proven useful in checking several authentication protocols (Anand and Saifulla (2023); Kumar and Hussain (2023); Hosseinzadeh et al. (2020); Seno et al. (2021)). Leveraging the Security Protocol Description Language (SPDL), Scyther evaluates the validity of protocol security claims. Moreover, it automates their generation and validation processes, ensuring a thorough protocol assessment.

Fig. 5 shows the implementation code of our proposed protocol in Scyther, which includes two roles: the IoT device D_j and the fog server F_i . The security claims of the roles within the protocol are described as follows (Rehman et al. (2021); Klevstad (2016)):

- **Secrecy:** This claim safeguards parameters that must be kept hidden from potential attackers. In our protocol, random numbers r_d, r_f are deemed confidential.
- **Session-Key Reveal (SKR):** This claim is used in the protocol to identify the session key SK and claim that it is secret.
- **Non-injective agreement (Niagree):** The claim ensures that both D_j and F_i agreed on the data values and verified the noninjective agreement, preventing any device injection attack.
- **Non-injective synchronization (Nisynch):** This claim considers the timestamps as nonces securely transmitted over public channels, thus preventing replay attacks.
- **Alive:** This claim ascertains the active participation of the counterparties in the protocol (D_j or F_i). If D_j completes a protocol run with F_i , then it confirms that F_i has previously initiated the protocol.
- **Weak agreement (Weakagree):** This claim ensures that if D_j successfully complete a protocol run with F_i , then F_i believes that it run recently the protocol with D_j . This claim confirms that our protocol provides identifier authentication and data source authentication using A_d, A_f , which helps to prevent impersonation and Man-in-the-Middle attacks.

Fig. 6 shows the verification results of LAAKA using the Scyther tool. If the status is *OK*, it implies the protocol satisfies the given property. Conversely, a *Fail* status denotes the protocol's failure to satisfy a specific security property.

6. Performance evaluation

This section investigates the security and efficiency of our scheme by comparing its performance with the performance of authentication

<pre> usertype TimeStamp; hashfunction H; secret XOR: Function; secret IDf, IDd, TIDd, TIDdRA, TIDfRA, rd, rf, r1, r2, K: Nonce; macro Af=H(IDf,r1); macro Ad=H(IDd,r2); macro Bk=H(Ad,Af,K); macro Ed=XOR(rd,H(Bk,Af)); protocol Proposed-Protocol(Fi,Dj){ role Fi{ fresh Tf,Ts:TimeStamp; var TIDd,Td,Cd,Ed,Gd,Ack; recv_1(Dj,Fi,(TIDd,Td,Cd,Ed,Gd)); macro rdstar=XOR(Ed,H(Bk,Af)); macro Cdstar=H(Td,rdstar); macro TIDdnewstar=XOR(TIDd,rdstar); macro Gdstar=H(Ad,TIDdnewstar,Bk,rdstar); match(Cd,Cdstar); match(Gd,Gdstar); macro Cf=H(Tf,rf); macro SK=H(rdstar,rf,Ts); macro Ef=XOR(rf,H(TIDdnewstar)); macro TIDfnew=XOR(TIDfRA,rf); macro Gf=H(TIDfnew,Bk,rf,SK,Ts); send_2(Fi,Dj,(TIDfRA,Tf,Ts,Cf,Ef,Gf)); recv_3(Dj,Fi,(Ack)); macro Ackf=H(rf,Bk,SK); match(Ack,Ackf); claim(Fi,Alive); claim(Fi,Nisynch); claim(Fi,Niagree); claim(Fi,Weakagree); claim(Fi,SKR,SK); claim(Fi,Secret,rf); claim(Fi,Secret,rdstar); claim(Fi,Secret,TIDd); }; </pre>	<pre> role Dj{ fresh Td:TimeStamp; macro Cd=H(Td,rd); macro Ed=XOR(rd,H(Bk,Af)); macro TIDnew=XOR(TIDdRA,rd); macro Gd=H(Ad,TIDnew,Bk,rd); send_1(Dj,Fi,(TIDdRA,Td,Cd,Ed,Gd)); var TIDf,Tf,Ts,Cf,Ef,Gf; recv_2(Fi,Dj,(TIDf,Tf,Ts,Cf,Ef,Gf)); macro rfstar=XOR(Ef,H(TIDnew)); macro Cfstar=H(Tf,rfstar); match(Cf,Cfstar); macro SKstar=H(rdstar,Ts); macro TIDfnewstar=XOR(TIDf,rfstar); macro Gfstar=H(TIDfnewstar,Bk,rfstar,SKstar,Ts); match(Gf,Gfstar); macro Ack=H(rfstar,Bk,SKstar); send_3(Dj,Fi,(Ack)); claim(Dj,Alive); claim(Dj,Nisynch); claim(Dj,Niagree); claim(Dj,Weakagree); claim(Dj,SKR,SKstar); claim(Dj,Secret,rfstar); claim(Dj,Secret,rd); claim(Dj,Secret,TIDf); }; </pre>
--	--

Fig. 5. Implementation code of LAAKA in Scyther.

Claim	Status	Comments
Proposed_Protocol_Fi	Alive	Ok
Proposed_Protocol_Fi2	Nisynch	Ok
Proposed_Protocol_Fi3	Niagree	Ok
Proposed_Protocol_Fi4	Weakagree	Ok
Proposed_Protocol_Fi5	SKR H(XOR(XOR(rd,H(H(IDd,r2),H(IDf,r1),K),H(IDf,...	Ok
Proposed_Protocol_Fi6	Secret rf	Ok
Proposed_Protocol_Fi7	Secret XOR(XOR(rd,H(H(IDd,r2),H(IDf,r1),K),H(IDf,...	Ok
Proposed_Protocol_Fi8	Secret TIDd	Ok
Dj Proposed_Protocol_Dj1	Alive	Ok
Proposed_Protocol_Dj2	Nisynch	Ok
Proposed_Protocol_Dj3	Niagree	Ok
Proposed_Protocol_Dj4	Weakagree	Ok
Proposed_Protocol_Dj5	SKR H(rdstar,XOR(rf,H(XOR(TIDd,XOR(XOR(rd,H(H(IDd,...	Ok
Proposed_Protocol_Dj6	Secret XOR(XOR(rf,H(XOR(TIDd,XOR(XOR(rd,H(H(IDd,...	Ok
Proposed_Protocol_Dj7	Secret rd	Ok
Proposed_Protocol_Dj8	Secret TIDf	Ok

Fig. 6. Verification results of LAAKA in Scyther.

and key agreement schemes presented by Weng et al. (2021), Jia et al. (2019), Ma et al. (2019), Hamada et al. (2022), Wu et al. (2021b), Rahman and Wen (2019), Wu et al. (2021a), Chen et al. (2021), Eftekhari et al. (2021), and Wazid et al. (2019a), in terms of security features, computational cost, and storage requirements.

6.1. Security features

Table 3 shows the security features provided by our scheme LAAKA compared with other introduced schemes. We use “Yes” to indicate that the security feature is provided by the corresponding scheme, whereas “No” indicates that the security feature is not provided by the scheme.

According to Table 3, all evaluated schemes, including ours, successfully implement mutual authentication and offer resistance to Man-in-the-Middle (MitM) attack. Weng et al. (2021)’s scheme is unique in its use of hash functions and bitwise XOR operations, yet it is the only one among them not resistant to replay attacks. Excluding Wazid et al. (2019a)’s scheme, all others effectively defend against impersonation

attacks targeting IoT devices and fog servers. However, the schemes introduced by Weng et al. (2021) and Wazid et al. (2019a) do not provide protection against key exposure attacks. Additionally, Perfect Forward Secrecy (PFS) is a feature in all the reviewed schemes except for those by Rahman and Wen (2019) and Eftekhari et al. (2021).

The schemes proposed by Hamada et al. (2022) and Rahman and Wen (2019) assume that the secret shared key is stored in clear form in the device’s memory, making them vulnerable to the unknown key-share attack and the device (i.e., IoT device, fog server) physical capture attack.

However, a common limitation of these schemes is the increased communication latency, attributed to the registration authority being located in the cloud layer, which is involved in device registration and authentication processes. In contrast, our proposed scheme (LAAKA) is more robust and reliable, offering greater resistance against a wider range of well-known attacks and enhanced security features.

6.2. Computational cost

The computational cost is computed by determining the number of the used cryptographic operations and their required computational time. The computational time of the operations is taken from (Hamada et al. (2022)) and shown in Table 4. The authors used a machine with CPU (Intel Pentium 2.20 GHz), RAM (2 GB), and OS (Ubuntu 12.04.1 LTS 32-bit).

We assume that the key derivation function used in (Rahman and Wen (2019)) is a *hmac* function, so it requires computational time as the hash function. We also assume that the time required for the bitwise XOR operation is negligible.

Table 5 presents a comparison of various schemes based on computational cost. Our scheme achieves the lowest computational cost of 0.0437ms. Weng et al. (2021)’s scheme, while offering a comparable minimal cost of 0.0529ms, has vulnerabilities that our scheme addresses, as discussed in section 6.1.

Jia et al. (2019)’s scheme leads to a higher cost of 33.0633ms due to its use of extensive computational operations like scalar multiplication and bilinear pairings. Wu et al. (2021b)’s scheme enhances Jia’s model but raises the computational cost to 39.7413ms by increasing scalar multiplications to ten.

Ma et al. (2019)’s scheme, while improving Jia et al. (2019)’s approach and eliminating bilinear pairings, results in a higher computational cost of 37.8857 due to increased scalar multiplications. This approach has been further refined by Eftekhari et al. (2021), who reduced the number of scalar multiplications to nine, consequently lowering the computational cost to 20.2366.

Using a fuzzy extractor with scalar multiplication results in lower costs compared to using bilinear pairings. The methods by Wazid et al. (2019a), Chen et al. (2021), and Wu et al. (2021a) demonstrate costs of 15.6982ms, 22.3106ms, and 24.5458ms, respectively.

Hamada et al. (2022) and Rahman and Wen (2019) show that using symmetric and asymmetric encryption can be more cost-effective than scalar multiplications and bilinear pairings, with costs of 22.9093ms and 11.7885ms, respectively.

However, the number of operations used in our scheme is illustrated as follows:

- IoT device D_j : There is no computational operation required from the device at the initialization. When it asks for registration, it performs one hash function to compute A_d and then receives TID_d, TID_f, A_f, B_k parameters to store them in the memory. In the authentication phase, the IoT device performs three hash functions to calculate C_d, E_d, G_d parameters in $Auth_{req}$, four hash functions once it receives $Auth_{rep}$ to extract r_f , verify both C_f and G_f , and calculate SK . It also performs one more hash function when it submits Ack to the fog server. Thus, the total number

Table 3
Security features comparison.

Security Feature	Weng et al. (2021)	Jia et al. (2019)	Ma et al. (2019)	Hamada et al. (2022)	Wu et al. (2021b)	Rahman and Wen (2019)	Wu et al. (2021a)	Chen et al. (2021)	Eftekhari et al. (2021)	Wazid et al. (2019a)	LAAGA
Mutual authentication	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Establish session key without involving registration authority	No	No	No	Yes	No	No	No	No	No	No	Yes
Immune to Man-in-The-Middle (MitM) attack	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Immune to replay Attack	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Immune to key exposure attack	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Immune to known session key attack	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Immune to IoT device impersonation attack	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Immune to fog server impersonation attack	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Immune to IoT device physical capture attack	Yes	Yes	Yes	No	Yes	No	Yes	Yes	Yes	Yes	Yes
Immune to fog server physical capture attack	Yes	Yes	Yes	No	Yes	No	Yes	Yes	Yes	Yes	Yes
Immune to IoT device traceability attack	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Immune to unknown key-share attack	Yes	Yes	Yes	No	Yes	No	Yes	Yes	Yes	Yes	Yes
Immune to Ephemeral-Secret-Leakage (ESL) attack	Yes	No	No	Yes	Yes	No	Yes	Yes	Yes	No	Yes
Perfect Forward Secrecy (PFS)	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes	No	Yes	Yes
Confidential communication session	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Immune to privileged insider attack	Yes	No	No	No	Yes	No	Yes	Yes	Yes	Yes	Yes
Identity Anonymity	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Low latency authentication	No	No	No	No	No	No	No	No	No	Yes	Yes
Lightweight	Yes	No	No	No	No	No	No	No	No	No	Yes

Table 4
Computational time of cryptographic operations.

Notation	Meaning	Computational Time (ms)
T_h	Hash/Hmac function time	0.0023
T_{sm}	ECC scalar multiplication time	2.226
T_{pa}	ECC point adding time	0.0288
T_{sed}	Symmetric encryption/decryption time	0.0046
T_{aed}	Asymmetric encryption/decryption time	3.85
T_s	Sign operation time	3.85
T_v	Signature verification time	0.1925
T_{fe}	Fuzzy extraction time	2.226
T_b	Bilinear pairing time	5.811

of hash functions performed by the IoT device is nine hash functions.

- Fog server F_i : At the registration phase, the fog server performs one hash function to compute A_f , while it performs seven hash functions at the authentication phase once it receives $Auth_{rep}$ to extract r_d , verify both C_d and G_d , calculate C_f, E_f, G_f, SK . Finally, the fog server performs one more hash function to verify the received message Ack . So, the total hash functions required by the fog server are nine functions.
- Registration authority RA : It performs only one hash function to calculate B_k at the registration phase.

Based on that, our scheme requires (19) hash functions as shown in Table 5, which makes it lightweight and suitable for limited resources devices such as IoT and fog devices.

6.3. Storage requirements

To calculate the storage requirements of our scheme to be compared with the other schemes, we suppose that the hash output length is 160 bits, the required bits for the random number, password, and identifier is 128, while the timestamp is only 32 bits, and the symmetric encryption key is 128 bits (Hamada et al. (2022); Rana et al. (2021b)). Using the elliptic curve $E(a, p)$, we suppose that the length of a and b is 8 bits, while p is 512 bits and q is 160 bits. The public and private keys length with ECC is 160 bits. Moreover, the length of the master secret key and generated certificate is also 160 bits (Hamada et al. (2022)). On the other hand, the length of the keys is 1024 bits with other public key cryptography (i.e., RSA) (Wazid et al. (2019b)). According to Hamada et al. (2022), the identifier length of the user is 32 bits, while the identifier of the fog area and fog server is 8 bits. We calculate the required bits for their scheme according to our assumptions. Therefore, the required number of bits differs from what they mentioned in their paper. To further clarify, we indicate the public and private key of the fog server as $Pub_f, Priv_f$, respectively, while the public and private key of the registration authority as $Pub_{RA}, Priv_{RA}$, respectively.

Table 5
Computational cost comparison.

Reference	IoT device/User	Fog Server	Registration Authority	Total Operations	Total Time (ms)
Weng et al. (2021)	$5T_h$	$5T_h$	$13T_h$	$23T_h$	0.0529
Jia et al. (2019)	$2T_{sm} + 6T_h + 1T_b$	$2T_{sm} + 4T_h + 1T_b$	$3T_{sm} + 11T_h + 1T_b$	$7T_{sm} + 21T_h + 3T_b$	33.0633
Ma et al. (2019)	$3T_{sm} + 4T_h$	$4T_{sm} + 4T_h$	$10T_{sm} + 11T_h$	$17T_{sm} + 19T_h$	37.8857
Hamada et al. (2022)	$2T_{sm} + 3T_{sed} + 2T_h$	$2T_{sm} + 3T_{sed} + 1T_{aed} + 1T_v + 1T_h$	$1T_{sm} + 1T_{aed} + T_s + 1T_h$	$5T_{sm} + 6T_{sed} + 2T_{aed} + 1T_s + 1T_v + 4T_h$	22.9093
Wu et al. (2021b)	$3T_{sm} + 7T_h + T_b$	$3T_{sm} + 4T_h + T_b$	$4T_{sm} + 10T_h + T_b$	$10T_{sm} + 21T_h + 3T_b$	39.7413
Rahman and Wen (2019)	$2T_{sed} + 6T_h$	$1T_{aed} + 1T_v + 4T_h$	$2T_{sed} + 1T_{aed} + 1T_s + 2T_h$	$4T_{sed} + 2T_{aed} + 1T_s + 1T_v + 12T_h$	11.7885
Wu et al. (2021a)	$2T_{sm} + 8T_h + T_{fe}$	$4T_{sm} + 5T_h$	$4T_{sm} + 13T_h$	$10T_{sm} + 26T_h + T_{fe}$	24.5458
Chen et al. (2021)	$2T_{sm} + 6T_h + T_{fe}$	$3T_{sm} + 12T_h$	$4T_{sm} + 4T_h$	$9T_{sm} + 22T_h + T_{fe}$	22.3106
Eftekhari et al. (2021)	$3T_{sm} + 11T_h + T_{pa}$	$3T_{sm} + 12T_h + T_{pa}$	$3T_{sm} + 15T_h + 2T_{pa}$	$9T_{sm} + 38T_h + 4T_{pa}$	20.2366
Wazid et al. (2019a)	$3T_{sm} + 22T_h + 2T_{fe}$	$2T_{sm} + 14T_h + T_{pa}$	$2T_h$	$5T_{sm} + 38T_h + 2T_{fe} + T_{pa}$	15.6982
LAACA	$9T_h$	$9T_h$	$1T_h$	$19T_h$	0.0437

In Weng et al. (2021)'s approach, they define MK as the master key. K_{cd} is the shared secret key between the edge device and the cloud, and K_{cf} is the shared secret key between the fog server and the cloud. Additionally, B_d and B_f are identified as secure hash values. Rahman and Wen (2019) and Hamada et al. (2022) consider other keys in their studies: K_{fu-RA} is the shared secret key between the fog user and the RA, K_{fu-f} is the shared secret key between the fog user and the fog server, and K_{fu} is identified as the secret key of the fog user. While ID_{fu} , ID_{fa} , and ID_f represent the identifiers of the fog user, fog area, and fog server, respectively.

Wu et al. (2021b) enhanced the scheme originally proposed by Jia et al. (2019). In both schemes, the RA maintains identifiers of IoT and fog devices, along with randomly generated numbers x_i, y_j . In Jia et al. (2019)'s approach, the IoT device stores a hashed value R_i provided by RA. This value is derived from the IoT device's password and identifier, combined with the random numbers x_i, s . Conversely, Wu et al. (2021b) introduce an additional parameter V_i to be stored alongside R_i . This parameter is derived from the IoT device's identifier, x_i, s , and another random number r_i . Regarding the fog server, in Jia et al. (2019)'s method, the server stores a hashed value R_j , also sent by the RA, which is calculated using the fog server's identifier and the random numbers y_j and s . On the other hand, Wu et al. (2021b) have the fog server store g_j , which is computed similarly to R_j .

In Ma et al. (2019)'s study, the cloud server holds the fog user's ID, ID_{fu} . It also stores two key components: $D_{IDi} = h(ID_{fu}||s)^P$ and $D_{IDj} = h(ID_f||s)^P$, which are respectively stored by the IoT device and the fog server. Here, the scalar output from the hash function is exponentiated with the generator point in Elliptic Curve Cryptography (ECC), where P is 512 bits. In the approach introduced by Eftekhari et al. (2021), different elements are used. Rnd_d and Rnd_f are random numbers. RID_{fu} , RID_f refers to hashed values derived from a random value, a secret value (such as the user's password or the fog server's secret key), and an identifier. The parameter RPW_{fu} is an XOR of a hashed value of the user's password and a random value shared between the user and RA, further hashed with the user's RID and the RA's private key. Likewise, R_{KEY} is an XOR of a hashed value of the fog server's secret key with a random value shared between the fog server and RA, and is then hashed with the fog server's RID and the RA's private key.

In the scheme proposed by Chen et al. (2021), the biometric parameter, BIO , has a length of 512 bits. The value SID_i represents the hashed value of the user identifier combined with a secret key s . The parameters $A_i, D_i, rsFSID_j, B_i$ are hash values, whereas X_j and r_f are random numbers. According to Wu et al. (2021a), A_i, F_i , and HP_i are also hash values. In the Wazid et al. (2019a)' scheme, TID_v and TID_f represent the temporary identifiers of the vehicular IoT device and the fog server, respectively. RID_v and RID_f are the hashed-based random identifiers of these devices, while k_{m1j} and k_{m2j} are hashed-based secret keys.

Table 6 shows that our approach is the most storage-efficient among the compared schemes, making it well-suited for devices with con-

strained resources, such as IoT devices and Fog servers. Its low storage requirement makes it particularly appropriate for environments where minimizing resource usage is crucial.

7. Conclusion and future work

This study introduced LAACA, a Lightweight Anonymous Authentication and Key Agreement scheme designed for the IoT-fog environment. LAACA ensures an optimal balance between lightweight operation and robust security, utilizing hash functions and bitwise XOR for mutual authentication between IoT devices and fog servers. Moreover, it effectively establishes secure session keys without relying on a trusted registration authority, situated within the fog layer to minimize latency. We have accurately designed our scheme to provide secure communication within the context of the DY threat model and improve the confidentiality of the session key under the CK threat model. Another key feature of LAACA is its emphasis on anonymity, ensuring user privacy by concealing device identities. These strengths have been proved through extensive formal (i.e., BAN, and ROM) and informal security analyses, demonstrating the protocol's effectiveness in preventing a wide array of potential security threats. Another software-based analysis has been done using the Scyther tool to confirm the robustness of our scheme against a wide range of attacks and vulnerabilities. Moreover, comparative evaluations with existing authentication schemes have illustrated our protocol's distinct advantages in terms of security feature set, computational overhead, and storage requisites. However, the research field is ever-evolving, and our efforts to refine and optimize IoT security remain ongoing.

In future work, we plan to enhance the robustness of our system by decentralizing the registration process. This strategy aims to reduce vulnerabilities linked to a single point of failure, thereby improving the overall network reliability. Moreover, addressing the mobility of IoT devices is a key focus; we plan to refine our scheme to effectively manage device movement. This enhancement will ensure consistent authentication integrity while preserving operational efficiency. Another future plan is extending the scheme to implement IoT users, incorporating advanced techniques like multi-factor and biometric verification methods. This expansion will also include inter-fog device-to-device (D2D) authentication, ensuring secure communication among all involved entities. Additionally, this scheme can be implemented and evaluated in a real testbed environment with actual resource-constrained IoT devices, local fog devices, and cloud server instances.

CRedit authorship contribution statement

Hala Ali: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Irfan Ahmed:** Supervision, Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Writing – original draft, Writing – review & editing.

Table 6
Storage Requirements comparison.

Reference	IoT device/User	Fog Server	Registration Authority	Total Required storage (bits)
Weng et al. (2021)	$ID_f, TID_d, B_d, h(MK \parallel K_{cd}) : 576$	$TID_f, B_f, h(MK \parallel K_{cf}) : 448$	$ID_d, ID_f, TID_d, TID_f, B_d, B_f, MK, K_{cd}, K_{cf} : 1312$	2336
Jia et al. (2019)	$ECC(a, b, p, q), R_i : 848$	$ECC(a, b, p, q), R_j : 848$	$ECC(a, b, p, q), ID_{fu}, ID_f, x_i, y_j : 1200$	2896
Ma et al. (2019)	$ECC(a, b, p, q), D_{ID_i} : 1200$	$ECC(a, b, p, q), D_{ID_j} : 1200$	$ECC(a, b, p, q), ID_{fu}, D_{ID_i} : 1328$	3728
Hamada et al. (2022)	$ECC(a, b, p, q), K_{fu}, ID_{fu} : 976$	$ECC(a, b, p, q), K_{fu-f}, ID_{fu}, ID_{fa}, ID_f, Cert, Pub_f, Priv_f, Pub_{RA} : 1872$	$ECC(a, b, p, q), K_{fu-f}, ID_{fu}, ID_{fa}, ID_f, Pub_{RA}, Priv_{RA}, Pub_f : 1712$	4560
Wu et al. (2021b)	$ECC(a, b, p, q), R_i, V_i : 1008$	$ECC(a, b, p, q), g_j : 848$	$ECC(a, b, p, q), ID_{fu}, ID_f, x_i, y_j : 1200$	3056
Rahman and Wen (2019)	$K_{fu-RA}, K_{fu-f}, ID_{fu}, ID_{fa}, ID_f, SK : 864$	$K_{fu-f}, ID_{fu}, ID_{fa}, ID_f, Pub_f, Priv_f, Pub_{RA}, SK : 3776$	$K_{fu-RA}, K_{fu-f}, ID_{fu}, ID_{fa}, ID_f, Pub_{RA}, Priv_{RA}, Pub_f : 3776$	8416
Wu et al. (2021a)	$ECC(a, b, p, q), BIO, A_i : 1360$	$ECC(a, b, p, q), TID_f, F_i : 976$	$ECC(a, b, p, q), H P_i, TID_d, ID_f, r_f : 1232$	3568
Chen et al. (2021)	$ECC(a, b, p, q), BIO, A_i, D_i : 1520$	$ECC(a, b, p, q), rsFSID_j, X_j : 976$	$ECC(a, b, p, q), SID_i, ID_f, B_i, r_f : 1264$	3760
Eftekhari et al. (2021)	$ECC(a, b, p, q), RPw_{fu}, Rnd_d, Pub_{RA} : 1136$	$ECC(a, b, p, q), RKey_f, Rnd_f, Pub_{RA} : 1136$	$ECC(a, b, p, q), ID_{fu}, ID_d, ID_f, RID_{fu}, RID_f : 1392$	3664
Wazid et al. (2019a)	$ECC(a, b, p, q), TID_v, RID_v, h(K_{m1j} \parallel RID_v) : 1136$	$ECC(a, b, p, q), TID_v, RID_v, TID_f, RID_f, K_{m1j}, k_{m2j}, Pub_{RA} : 1744$	$ECC(a, b, p, q), TID_v, RID_v, TID_f, RID_f, K_{m1j}, k_{m2j}, h(K_{m1j} \parallel RID_v) : 1744$	4624
LAACA	$TID_d, TID_f, A_f, B_k : 576$	$TID_d, TID_f, A_d, B_k : 576$	$TID_d, TID_f, A_d, A_f, B_k, K : 896$	2048

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Abdussami, Mohammad, Amin, Ruhul, Vollala, Satyanarayana, 2022. Lassi: a lightweight authenticated key agreement protocol for fog-enabled iot deployment. *Int. J. Inf. Secur.* 21 (6), 1373–1387.
- AbuTaha, Mohammed, Farajallah, Mousa, Tahboub, Radwan, Odeh, Mohammad, 2011. Survey paper: cryptography is the science of information security. *Int. J. Comput. Sci. Secur. (IJCSS)*.
- Ali, Hala S., Sridevi, R., 2022. Credential-based authentication mechanism for iot devices in fog-cloud computing. In: *ICT Analysis and Applications*. Springer, pp. 307–318.
- Ali, Hala S., Sridevi, R., 2023. Mobility and security aware real-time task scheduling in fog-cloud computing for IoT devices: a fuzzy-logic approach. *Comput. J.*, bxad019. Oxford University Press.
- Ali, Hala S., Ranjan Rout, Rashmi, Parimi, Priyanka, Das, Sajal K., 2021a. Real-time task scheduling in fog-cloud computing framework for iot applications: a fuzzy logic based approach. In: *2021 International Conference on COMMunication Systems & NETWORKS (COMSNETS)*. IEEE, pp. 556–564.
- Ali, Zeeshan, Ashraf Chaudhry, Shehzad, Mahmood, Khalid, Garg, Sahil, Lv, Zhihan, Zikria, Yousaf Bin, 2021b. A clogging resistant secure authentication scheme for fog computing services. *Comput. Netw.* 185, 107731.
- Almurisi, Nasr, Tadisetty, Srinivasulu, 2022. Cloud-based virtualization environment for iot-based wsn: solutions, approaches and challenges. *J. Ambient Intell. Humaniz. Comput.* 13, 1–23.
- Amor, Arij Ben, Abid, Mohamed, Meddeb, Aref, 2017. A privacy-preserving authentication scheme in an edge-fog environment. In: *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. IEEE, pp. 1225–1231.
- Anand, Nimalakanti, Saifulla, M.A., 2023. En-lakp: lightweight authentication and key agreement protocol for emerging networks. *IEEE Access* 11, 28645–28657.
- Basin, David, Cremers, Cas, 2014. Know your enemy: compromising adversaries in protocol analysis. *ACM Trans. Inf. Syst. Secur.* 17 (2), 1–31.
- Bellare, Mihir, Rogaway, Phillip, 1993. Entity authentication and key distribution. In: *Annual International Cryptology Conference*. Springer, pp. 232–249.
- Bellare, Mihir, Rogaway, Phillip, 2005. Introduction to modern cryptography.
- Bellare, Mihir, Pointcheval, David, Rogaway, Phillip, 2000. Authenticated key exchange secure against dictionary attacks. In: *International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, pp. 139–155.

- Burrows, Michael, Abadi, Martin, Needham, Roger Michael, 1989. A logic of authentication. *Proc. R. Soc. Lond. Ser. A, Math. Phys. Sci.* 426 (1871), 233–271.
- Canetti, Ran, Krawczyk, Hugo, 2001. Analysis of key-exchange protocols and their use for building secure channels. In: *International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, pp. 453–474.
- Chen, Chien-Ming, Huang, Yanyu, Wang, King-Hang, Kumari, Saru, Wu, Mu-En, 2021. A secure authenticated and key exchange scheme for fog computing. *Enterp. Inf. Syst.* 15 (9), 1200–1215.
- Chen, Chien-Ming, Chen, Zhaoting, Kumari, Saru, Lap-ihot, Meng-Chang Lin, 2022. A lightweight authentication protocol for the Internet of health things. *Sensors* 22 (14), 5401.
- Cremers, Cas JF, 2008. The scyther tool: verification, falsification, and analysis of security protocols: tool paper. In: *International Conference on Computer Aided Verification*. Springer, pp. 414–418.
- Das, Ashok Kumar, Wazid, Mohammad, Reddy Yannam, Animi, Rodrigues, Joel JPC, Park, Youngho, 2019. Provably secure ecc-based device access control and key agreement protocol for iot environment. *IEEE Access* 7, 55382–55397.
- Das, Ashok Kumar, Bera, Basudeb, Wazid, Mohammad, Jamal, Sajjad Shaukat, Park, Youngho, 2021. On the security of a secure and lightweight authentication scheme for next generation iot infrastructure. *IEEE Access* 9, 71856–71867. <https://doi.org/10.1109/ACCESS.2021.3079312>.
- Dhillon, Parwinder Kaur, Kalra, Sheetal, 2017. Secure multi-factor remote user authentication scheme for Internet of things environments. *Int. J. Commun. Syst.* 30 (16), e3323.
- Dolev, Danny, Yao, Andrew, 1983. On the security of public key protocols. *IEEE Trans. Inf. Theory* 29 (2), 198–208.
- Eftekhari, Seyed Abdolreza, Nikooghadam, Morteza, Rafighi, Masoud, 2021. Security-enhanced three-party pairwise secret key agreement protocol for fog-based vehicular ad-hoc communications. *Veh. Commun.* 28, 100306.
- Gope, Proasanta, 2019. Laap: lightweight anonymous authentication protocol for d2d-aided fog computing paradigm. *Comput. Secur.* 86, 223–237.
- Günther, Christoph G., 1990. An identity-based key-exchange protocol. In: *Advances in Cryptology—EUROCRYPT'89: Workshop on the Theory and Application of Cryptographic Techniques Houthalen, Belgium, April 10–13, 1989 Proceedings* 8. Springer, pp. 29–37.
- Guo, Yimin, Guo, Yajun, 2021. Fogha: an efficient handover authentication for mobile devices in fog computing. *Comput. Secur.* 108, 102358.
- Guo, Yimin, Zhang, Zhenfeng, Guo, Yajun, 2020. Fog-centric authenticated key agreement scheme without trusted parties. *IEEE Syst. J.* 15 (4), 5057–5066.
- Hamada, Mariam, Salem, Sameh A., Salem, Fatty M., 2022. Lamas: lightweight anonymous mutual authentication scheme for securing fog computing environments. *Ain Shams Eng. J.* 13 (6), 101752.
- Hosseinazadeh, Mehdi, Ahmed, Omed Hassan, Ahmed, Sarkar Hasan, Trinh, Cuong, Bagheri, Nasour, Kumari, Saru, Lansky, Jan, Huynh, Bao, 2020. An enhanced authentication protocol for rfid systems. *IEEE Access* 8, 126977–126987.
- Ibrahim, Maged Hamada, 2016. Octopus: an edge-fog mutual authentication scheme. *Int. J. Netw. Secur.* 18 (6), 1089–1101.

- Jan, Saeed Ullah, Abbasi, Irshad Ahmed, Alqarni, Mohammed A., 2022. Lmas-shs: a lightweight mutual authentication scheme for smart home surveillance. *IEEE Access* 10, 52791–52803.
- Jia, Xiaoying, He, Debiao, Kumar, Neeraj, Choo, Kim-Kwang Raymond, 2019. Authenticated key agreement scheme for fog-driven iot healthcare system. *Wirel. Netw.* 25, 4737–4750.
- Jiang, Jiafu, Tang, Linyu, Gu, Ke, Jia, WeiJia, 2020. Secure computing resource allocation framework for open fog computing. *Comput. J. (ISSN 0010-4620)* 63 (4), 567–592.
- Kalaria, Rudri, Kayes, A.S.M., Rahayu, Wenny, Pardede, Eric, 2021. A secure mutual authentication approach to fog computing environment. *Comput. Secur.* 111, 102483.
- Kaur, Jasleen, Agrawal, Alka, Khan, Raees Ahmad, 2020. Security issues in fog environment: a systematic literature review. *Int. J. Wirel. Inf. Netw.* 27 (3), 467–483.
- Klevstad, Eirik, 2016. Security and key establishment in iee 802.15.4. Master's thesis. NTNU.
- Koblitz, Neal, Menezes, Alfred J., 2015. The random oracle model: a twenty-year retrospective. *Des. Codes Cryptogr.* 77, 587–610.
- Kumar, Manoj, Hussain, S.Z., 2023. An efficient and secure mutual authentication protocol in wireless body area network. *EAI Endorsed Trans. Perv. Health Tech.* 9.
- Kunal, Sourav, Saha, Arijit, Amin, Ruhul, 2019. An overview of cloud-fog computing: architectures, applications with security challenges. *IEEE Secur. Priv.* 2 (4), e72.
- Lansky, Jan, Sadrihojaji, Mahyar, Masoud Rahmani, Amir, Malik, Mazhar Hussain, Kazemian, Faeze, Hosseinzadeh, Mehdi, 2022. Development of a lightweight centralized authentication mechanism for the Internet of things driven by fog. *Mathematics* 10 (22), 4166.
- Li, Sensen, Zhang, Tikui, Yu, Bin, He, Kuan, 2020. A provably secure and practical puf-based end-to-end mutual authentication and key exchange protocol for iot. *IEEE Sens. J.* 21 (4), 5487–5501.
- Ma, Mimi, He, Debiao, Wang, Huaqun, Kumar, Neeraj, Choo, Kim-Kwang Raymond, 2019. An efficient and provably secure authenticated key agreement protocol for fog-based vehicular ad-hoc networks. *IEEE Int. Things J.* 6 (5), 8065–8075.
- Masud, Mehedi, Singh Gaba, Gurjot, Choudhary, Karanjeet, Shamim Hossain, M., Alhamid, Mohammed F., Muhammad, Ghulam, 2021. Lightweight and anonymity-preserving user authentication scheme for iot-based healthcare. *IEEE Int. Things J.* 9 (4), 2649–2656.
- Mukherjee, Mithun, Matam, Rakesh, Shu, Lei, Maglaras, Leandros, Ferrag, Mohamed Amine, Choudhury, Nikumani, Kumar, Vikas, 2017. Security and privacy in fog computing: challenges. *IEEE Access* 5, 19293–19304.
- Nandy, Tarak, Idna Bin Idris, Mohd Yamani, Noor, Rafidah Md, Kiah, Laiha Mat, Lun, Lau Sian, Juma'at, Nor Badrul Annur, Ahmedy, Ismail, Ghani, Norjihan Abdul, Bhat-tacharya, Sananda, 2019. Review on security of internet of things authentication mechanism. *IEEE Access* 7, 151054–151089.
- Ometov, Aleksandr, Liombe Molua, Oliver, Komarov, Mikhail, Nurmi, Jari, 2022. A survey of security in cloud, edge, and fog computing. *Sensors* 22 (3), 927.
- Panda, Suryakanta, Mondal, Samrat, Kumar, Neeraj, 2022. Slap: a secure and lightweight authentication protocol for machine-to-machine communication in industry 4.0. *Comput. Electr. Eng.* 98, 107669.
- Patonico, Simone, Braeken, An, Steenhaut, Kris, 2019. Identity-based and anonymous key agreement protocol for fog computing resistant in the Canetti-Krawczyk security model. *Wirel. Netw.*, 1–13.
- Rahman, Gohar, Wen, Chuah Chai, 2019. Mutual authentication security scheme in fog computing. *Int. J. Adv. Comput. Sci. Appl.* 10 (11).
- Rana, Minahil, Shafiq, Akasha, Altaf, Izwa, Alazab, Mamoun, Mahmood, Khalid, Chaudhry, Shehzad Ashraf, Zikria, Yousaf Bin, 2021a. A secure and lightweight authentication scheme for next generation iot infrastructure. *Comput. Commun.* 165, 85–96.
- Rana, Saurabh, Mishra, Dheerendra, Arora, Riya, 2021b. Privacy-preserving key agreement protocol for fog computing supported Internet of things environment. *Wirel. Pers. Commun.* 119 (1), 727–747.
- Rehman, Ubaid Ur, Park, Seong-Bae, Lee, Sungyoung, 2021. Secure health fog: a novel framework for personalized recommendations based on adaptive model tuning. *IEEE Access* 9, 108373–108391.
- Roman, Rodrigo, Lopez, Javier, Mambo, Masahiro, 2018. Mobile edge computing, fog et al.: a survey and analysis of security threats and challenges. *Future Gener. Comput. Syst.* 78, 680–698.
- Seno, Seyed Amin Hosseini, Nikooghadam, Mahdi, Budiarto, Rahmat, 2021. An efficient lightweight authentication and key agreement protocol for patient privacy. *Comput. Mater. Continua* 69 (3).
- Singh, Sunakshi, Chaurasiya, Vijay Kumar, 2021. Mutual authentication scheme of iot devices in fog computing environment. *Clust. Comput.* 24 (3), 1643–1657.
- Sowjanya, K., Dasgupta, Mou, Ray, Sangram, 2020. An elliptic curve cryptography based enhanced anonymous authentication protocol for wearable health monitoring systems. *Int. J. Inf. Secur.* 19 (1), 129–146.
- Suganthi, S.D., Anitha, R.V.S.S., Sureshkumar, Venkatasamy, Harish, S., Agalya, S., 2020. End to end light weight mutual authentication scheme in iot-based healthcare environment. *J. Reliab. Intell. Environ.* 6, 3–13.
- Tu, Shanshan, Waqas, Muhammad, Ur Rehman, Sadaqat, Aamir, Muhammad, Ur Rehman, Obaid, Jianbiao, Zhang, Chang, Chin-Chen, 2018. Security in fog computing: a novel technique to tackle an impersonation attack. *IEEE Access* 6, 74993–75001.
- Vinoth, R., Deborah, Lazarus Jegatha, Vijayakumar, Pandi, Kumar, Neeraj, 2020. Secure multifactor authenticated key agreement scheme for industrial iot. *IEEE Int. Things J.* 8 (5), 3801–3811.
- Wang, Lin, An, Haonan, Chang, Zhuo, 2020. Security enhancement on a lightweight authentication scheme with anonymity fog computing architecture. *IEEE Access* 8, 97267–97278.
- Wazid, Mohammad, Bagga, Palak, Kumar Das, Ashok, Shetty, Sachin, Rodrigues, Joel JPC, Akm-iov, Youngho Park, 2019a. Authenticated key management protocol in fog computing-based Internet of vehicles deployment. *IEEE Int. Things J.* 6 (5), 8804–8817.
- Wazid, Mohammad, Kumar Das, Ashok, Kumar, Neeraj, Vasilakos, Athanasios V., 2019b. Design of secure key management and user authentication scheme for fog computing services. *Future Gener. Comput. Syst.* 91, 475–492.
- Wazid, Mohammad, Kumar Das, Ashok, Bhat, Vivekananda, Lam-ciot, Athanasios V. Vasilakos, 2020. Lightweight authentication mechanism in cloud-based iot environment. *J. Netw. Comput. Appl.* 150, 102496.
- Weng, Chi-Yao, Li, Chun-Ta, Chen, Chin-Ling, Lee, Cheng-Chi, Deng, Yong-Yuan, 2021. A lightweight anonymous authentication and secure communication scheme for fog computing services. *IEEE Access* 9, 145522–145537.
- Wu, Tsu-Yang, Lee Lei Yang, Zhiyuan, Luo, Jia-Ning, Tso, Raylin, 2021a. Provably secure authentication key exchange scheme using fog nodes in vehicular ad hoc networks. *J. Supercomput.* 77, 6992–7020.
- Wu, Tsu-Yang, Wang, Tao, Lee, Yu-Qi, Zheng, Weimin, Kumari, Saru, Kumar, Sachin, 2021b. Improved authenticated key agreement scheme for fog-driven iot healthcare system. *Secur. Commun. Netw.* 2021, 1–16.
- Zhang, Jing, Zhong, Hong, Cui, Jie, Xu, Yan, Smaka, Lu Liu, 2020. Secure many-to-many authentication and key agreement scheme for vehicular networks. *IEEE Trans. Inf. Forensics Secur.* 16, 1810–1824.



Hala Ali: is a PhD student at Virginia Commonwealth University (VCU), Richmond, Virginia, USA. Her research interests include Cybersecurity, Cyber-Physical System Security, IoT, and Fog Computing. She received her B.Tech degree in Informatics Engineering from Al-Baath University, Homs, Syria in 2016, and an M.Tech degree in Computer Science and Information Security from the National Institute of Technology Warangal, Telangana, India, in 2020.



Irfan Ahmed: is an Associate Professor of Computer Science at Virginia Commonwealth University (VCU), Richmond, Virginia, USA. He is the Director of the Security and Forensics Engineering (SAFE) Research Lab. He is also a faculty fellow of VCU Cybersecurity Center. Before VCU, Ahmed was a Canizaro-Livingston Endowed Assistant Professor in Cybersecurity at the University of New Orleans (UNO), New Orleans, LA. His research interests are broadly in the area of cybersecurity, currently focusing on digital forensics, malware, cyber-physical systems, and cybersecurity education. Ahmed is a recipient of the ORAU Ralph E. Powe Junior Faculty Enhancement Award, an Outstanding Research Award from the American Academy of Forensic Sciences (AAFS), and the UNO's Early Career Research Prize. His research group has received three Best Paper Awards at DFRWS'20, ISC'13, and ICRC'11, and an Outstanding Poster Award at CODASPY'16.